

YourSport



**Corso di Laurea Magistrale in Ingegneria
Informatica
2025/2026**

Ingegneria del Software

**Autori:
Antonino Caruso
Davide Consoli
Pietro Alberio**

Sommario

1. Ideazione e analisi dei requisiti.....	3
1.1 Introduzione.....	3
1.2 Scopo del progetto.....	3
1.3 Posizionamento	3
1.4 Attori del Sistema.....	3
1.6 Obiettivi e casi d'uso.....	4
1.7 Specifiche supplementari.....	5
1.8 Glossario	6
2. Analisi Delle Iterazioni	7
Introduzione.....	7
Iterazione 1.....	7
UC2: prenotazione autonoma struttura	8
Iterazione 2	17
UC3: Gestione dei costi di prenotazione.....	17
Iterazione 3.....	26
UC1: Registrazione alla piattaforma	26
UC4: Gestione delle proprie prenotazioni	32
Iterazione 4.....	38
UC5: Segnalazione problemi struttura.....	38
UC6: Gestione segnalazioni.	43
UC7: Sospensione e Riattivazione Struttura.....	48
Iterazione 5.....	54
UC8: Visualizza le notifiche.	54
UC9: Visualizza statistiche incassi previsti:.....	58
UC10: Gestione Profilo Personale:	62
3. Diagramma delle Classi	68
4. Conclusioni e implementazioni future	68
4.1 Sintesi del progetto	68
4.2 Implementazioni future.....	69

1. Ideazione e analisi dei requisiti

1.1 Introduzione

La fase di ideazione è stata cruciale per gettare le basi di un progetto solido e ben strutturato. Lo scopo principale è stato sviluppare una chiara visione d'insieme, valutando preventivamente la fattibilità tecnica e le risorse necessarie per la realizzazione. Questa analisi preliminare è essenziale per prevenire criticità future. Per acquisire una comprensione dettagliata delle esigenze di sistema, il lavoro si è basato sull'analisi di strumenti fondamentali quali il Documento di Visione, il Modello dei Casi d'Uso e il Glossario

1.2 Scopo del progetto

YourSport è una piattaforma software ideata per la gestione automatizzata delle strutture sportive. L'obiettivo principale del sistema è fornire un ambiente centralizzato che permetta agli utenti di prenotare le risorse in autonomia, garantendo allo stesso tempo agli amministratori degli strumenti per il controllo delle tariffe, la gestione delle emergenze (inagibilità) e il tracciamento dei guasti strutturali. Il sistema è progettato per prevenire attivamente conflitti di prenotazione e per automatizzare le comunicazioni di disservizio.

1.3 Posizionamento

Opportunità di business: Il Software Gestionale YourSport si propone di sostituire le attuali metodologie di gestione non automatizzate adottate dagli impianti sportivi multiservizi. Implementando questa soluzione, si assicurerà la massima efficienza nell'amministrazione dei servizi richiesti dall'utente finale, garantendo al contempo una semplice e immediata fruibilità delle varie risorse strutturali.

Formulazione del problema: La gestione manuale dei processi costituisce un fattore limitante per la produttività amministrativa, incidendo negativamente sul monitoraggio in tempo reale delle prenotazioni. Di conseguenza, si manifestano elevati attriti operativi e rischi di errore nelle operazioni di reperimento, persistenza e update dei dati relativi agli utenti.

Formulazione della posizione del prodotto: Il software è rivolto a tutti gli impianti sportivi che non dispongono di un'automatizzazione della gestione delle risorse.

1.4 Attori del Sistema

Utente Registrato (sportivo): Colui che prenota e utilizza le strutture.

Admin: Colui che gestisce le strutture.

Sistema: Controlla scadenze o aggiorna i prezzi in base all'orario.

1.5 Requisiti

Il sistema deve possedere le seguenti caratteristiche:

- Registrazione alla piattaforma da parte dello sportivo

- Prenotazione autonoma di una struttura (o più strutture) o posto specificando giorno e fascia oraria da parte dello sportivo
- Gestione delle prenotazioni (modifica, visualizzazione ed eliminazione) da parte dello sportivo
- Gestione dei costi di prenotazione delle strutture da parte dell'admin (in base alle caratteristiche della struttura)
- Segnalazione di problemi relativi alla struttura da parte dello sportivo
- Gestione delle segnalazioni ricevute e amministrazione generale delle risorse
- Visualizzazione degli incassi previsti
- Gestione del profilo
- Visualizzazione notifiche
- Ricerca avanzata delle strutture basata su caratteristiche fisiche (copertura, terreno, ecc.)

1.6 Obiettivi e casi d'uso

Analizzando i requisiti e la descrizione delle parti interessate, sono stati individuati gli attori principali (Utente Registrato/Sportivo, Admin) e i relativi obiettivi. Da queste informazioni sono stati ricavati i casi d'uso principali.

Sportivo	Inserisce i propri dati per creare un account e accedere ai servizi.	UC1: Registrazione alla piattaforma
Sportivo	Vuole assicurarsi la disponibilità di una o più strutture/posti in una data/ora specifica.	UC2: Prenotazione autonoma struttura
Admin	Vuole aggiornare i prezzi delle risorse.	UC3: Gestione dei costi di prenotazione
Sportivo	Accede alla propria area personale per visualizzare e gestire l'elenco e lo stato delle prenotazioni.	UC4: Gestione delle proprie prenotazioni.
Sportivo	Riscontra un problema durante l'utilizzo dell'impianto e lo segnala.	UC5: Segnalazione problemi struttura
Admin	Visualizza le segnalazioni e ne aggiorna lo stato operativo.	UC6: Gestione segnalazioni

Admin	A fronte di disservizi ne notifica la chiusura agli sportivi interessati e annulla le prenotazioni.	UC7: Notifica impossibilità uso struttura per disservizio.
Sportivo	Visualizza le notifiche riguardanti le prenotazioni e i disservizi.	UC8: Visualizza le notifiche.
Admin	Consulta i dati relativi agli introiti delle prenotazioni.	UC9: Visualizza statistiche incassi previsti.
Sportivo	Accede alla propria area riservata per gestire i propri dati anagrafici.	UC10: Gestione Profilo Personale.

1.7 Specifiche supplementari

1.7.1 Usabilità:

Il sistema tramite l'interfaccia a riga di comando offre un'interazione chiara e differenziata in base al ruolo dell'utente loggato (Sportivo o Amministratore). Per lo Sportivo, il processo di ricerca, filtraggio e prenotazione delle strutture è fluido e restituisce feedback immediati e comprensibili (es. messaggi di errore chiari se una risorsa è già occupata o se un input non è valido). Per l'Amministratore, la gestione (modifica tariffe, sospensione strutture,...) permette di eseguire operazioni complesse con pochi passaggi.

1.7.2 Affidabilità e robustezza:

Il sistema deve prevenire attivamente l'overbooking e la corruzione dei dati. La robustezza è garantita da una validazione rigorosa degli input (es. il rigetto di ID inesistenti, il controllo sulle date degli eventi o il blocco delle email duplicate in fase di registrazione). L'affidabilità è inoltre assicurata dal meccanismo di persistenza tramite file JSON, che permette il salvataggio e il caricamento automatico dello stato globale per evitare perdite di dati in caso di arresto anomalo del programma.

1.7.3 Vincoli hardware e software:

Essendo YourSport un'applicazione sviluppata con paradigma Object-Oriented in linguaggio Java, il vincolo software è l'installazione di un ambiente di esecuzione Java sulla macchina host (Java Virtual Machine). Non sono richiesti requisiti hardware stringenti (CPU, RAM o GPU di fascia alta): il sistema è progettato per essere "leggero" e può essere eseguito su qualsiasi PC o server commerciale standard dotato di spazio su disco sufficiente per ospitare i file di salvataggio JSON.

1.7.4 Aspetti legali e licenza:

Trattandosi di un sistema che richiede la registrazione, **YourSport** acquisisce e memorizza dati sensibili degli utenti (Nome, Cognome, Email, Password). Il software deve quindi operare nel rispetto delle normative base sulla privacy (GDPR), garantendo che i dati anagrafici e lo storico delle prenotazioni non siano accessibili da utenti non autorizzati e che

l'incapsulamento protegga le credenziali. Essendo un progetto di origine accademica/didattica, il software può essere idealmente rilasciato sotto licenza Open Source, permettendone lo studio e l'utilizzo senza fini di lucro.

1.8 Glossario

- **YourSport (Sistema):** La piattaforma software gestionale sviluppata per automatizzare le prenotazioni, la manutenzione e l'amministrazione di un complesso di impianti sportivi.
- **Utente:** L'entità generale che rappresenta un individuo registrato nel sistema. Da questa astrazione derivano i due ruoli applicativi concreti: lo Sportivo e l'Amministratore.
- **Sportivo:** L'Attore primario del sistema. È l'utente finale (cliente) che accede all'applicazione per ricercare strutture, confermare o annullare prenotazioni, aggiornare la propria anagrafica e aprire ticket per la segnalazione di guasti.
- **Amministratore (Admin):** L'Attore con privilegi di sistema elevati. Rappresenta il gestore dell'impianto sportivo. Ha il compito di impostare le regole economiche (Tariffe), monitorare le segnalazioni di guasto, sospendere le strutture inagibili e visualizzare le metriche finanziarie (Statistiche).
- **Struttura (o Risorsa):** L'impianto fisico o l'area sportiva messa a disposizione dal catalogo. Ogni struttura è definita da un ID univoco, uno sport di riferimento, una lista di caratteristiche, una capienza massima e un costo base. Può essere fruita in due modalità: come Struttura Intera o a Posto Singolo.
- **Struttura Intera (Uso Esclusivo):** Modalità di fruizione in cui un impianto (es. Campo da Tennis) viene affittato e bloccato nella sua totalità. Una volta effettuata la prenotazione per una specifica fascia oraria, il sistema rifiuta qualsiasi altra richiesta di accesso simultaneo da parte di terzi.
- **Posto Singolo (Uso Condiviso):** Singola unità di accesso per impianti a fruizione collettiva (es. Piscina). In questa modalità gli utenti non prenotano l'intero ambiente, ma dichiarano la quantità di "posti" o "ingressi" che desiderano occupare. Il sistema accetta prenotazioni parallele per la stessa fascia oraria fino all'esaurimento della capienza massima.
- **Prenotazione:** Il contratto temporale stipulato tra il Sistema e uno Sportivo. Rappresenta l'assegnazione di una Struttura Intera o di un certo numero di Posti Singoli per una data e una fascia oraria specifiche, a fronte di un costo calcolato.

- **Tariffa:** La politica matematica di calcolo dei prezzi associata a una Struttura. Si divide in:
- **Tariffa ORARIO:** Applica il costo base in base alle ore di affitto, tipica per le strutture a uso esclusivo.
- **Tariffa PERSONA:** Moltiplica il costo base per la quantità di posti singoli prenotati, tipica per le strutture a uso condiviso.
- **Stato Operativo (Inagibilità):** Valore logico che indica se una Struttura è fisicamente utilizzabile. Se un Admin imposta lo stato su "Sospesa" (Fuori Servizio), la struttura viene temporaneamente chiusa alle prenotazioni.
- **Notifica:** Un messaggio di sistema generato automaticamente e indirizzato a uno specifico Sportivo, utilizzato tipicamente per comunicare l'annullamento forzato di una prenotazione a causa della sospensione improvvisa di una Struttura.
- **Segnalazione:** Un avviso testuale creato da uno Sportivo per segnalare un'anomalia o un danno fisico all'interno di una Struttura.
- **Stato della Segnalazione:** L'indicatore di avanzamento della risoluzione di un guasto. Può assumere i valori "Aperta" (appena inviata dallo Sportivo), "In lavorazione" (presa in carico dall'Admin), "Risolta" (guasto riparato) o "Rifiutata" (falsa segnalazione).

2. Analisi Delle Iterazioni

Introduzione

Utilizzando l'approccio iterativo, l'architettura del sistema YourSport è sviluppata in maniera incrementale, aggiungendo nuove funzionalità e raffinandone le precedenti ad ogni iterazione, nello specifico la realizzazione è stata articolata su 5 iterazioni.

Iterazione 1

Nella prima iterazione abbiamo gettato le fondamenta dell'intero sistema **YourSport**, concentrandoci sullo sviluppo del nucleo commerciale dell'applicazione: il **Caso d'Uso 2**

Per garantire un'architettura solida, coesa e a basso accoppiamento, la progettazione è stata guidata dai seguenti pattern:

- **Singleton (GoF):** Applicato alla classe principale YourSport per garantire che esista un'unica istanza globale del sistema in memoria, fungendo da registro centrale.

- **Facade Controller (GRASP):** Abbiamo assegnato a :YourSport il ruolo di coordinatore unico. È il primo oggetto che riceve le chiamate dall'interfaccia e le smista all'interno del dominio.
- **Creator (GRASP):** Abbiamo delegato a :YourSport la responsabilità di istanziare i nuovi oggetti Prenotazione, aggregando i dati di input.
- **Information Expert (GRASP):** Abbiamo distribuito l'intelligenza alle classi di dominio. Ad esempio, l'oggetto Prenotazione è l'esperto designato per calcolare il proprio costo totale, interrogando l'oggetto Struttura per conoscerne le regole tariffarie.

UC2: prenotazione autonoma struttura

Nome caso d'uso	UC2: Prenotazione autonoma struttura
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Sportivo (Utente registrato)
Parti interessate e interessi	- Sportivo: vuole assicurarsi la disponibilità di una struttura in una data/ora specifica
Pre-condizioni	Lo sportivo ha effettuato il login al sistema
Garanzie di successo	La prenotazione viene registrata nel sistema e lo sportivo ne riceve la conferma

Scenario principale di successo	1. Lo Sportivo accede alla sezione di prenotazione. 2. Lo Sportivo effettua una ricerca filtrata della struttura con le caratteristiche desiderate 3. Lo Sportivo seleziona la tipologia di struttura desiderata 4. Il Sistema mostra le disponibilità aggiornate in tempo reale (verificando i criteri di ricerca, l'occupazione della risorsa o la capienza residua dei posti). 5. Lo Sportivo seleziona una o più risorse per il giorno e la fascia oraria desiderata 6. Il Sistema calcola il costo complessivo delle risorse selezionate 7. Lo Sportivo conferma la prenotazione e blocca tutte le risorse scelte. 8. Il Sistema registra la prenotazione e aggiorna la disponibilità della risorsa (decrementando i posti disponibili o bloccando l'orario). 9. Lo Sportivo riceve la conferma della prenotazione effettuata
Estensioni	*a. In qualsiasi momento il sistema fallisce e ha un arresto improvviso. 1. L'admin riavvia il software e ripristina lo stato precedente del sistema. 2. Il sistema ripristina lo stato. 4a. La fascia oraria scelta non è più disponibile (conflitto in tempo reale). 1. Il Sistema notifica che la risorsa è già occupata. 2. Lo Sportivo ripete il passo 4 scegliendo una nuova data/ora. 6a. Problema durante il salvataggio dei dati (errore update). 1. Il Sistema segnala un errore tecnico. 2. La transazione viene annullata e lo stato ripristinato.

Modello di dominio

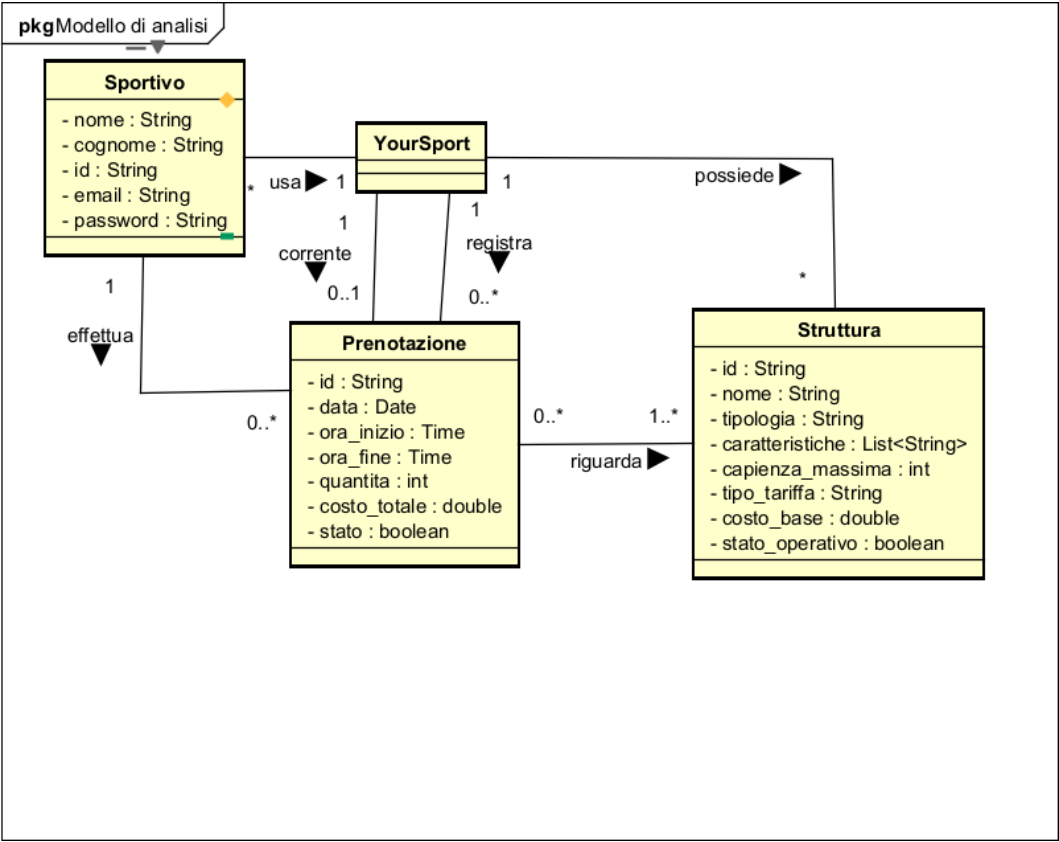
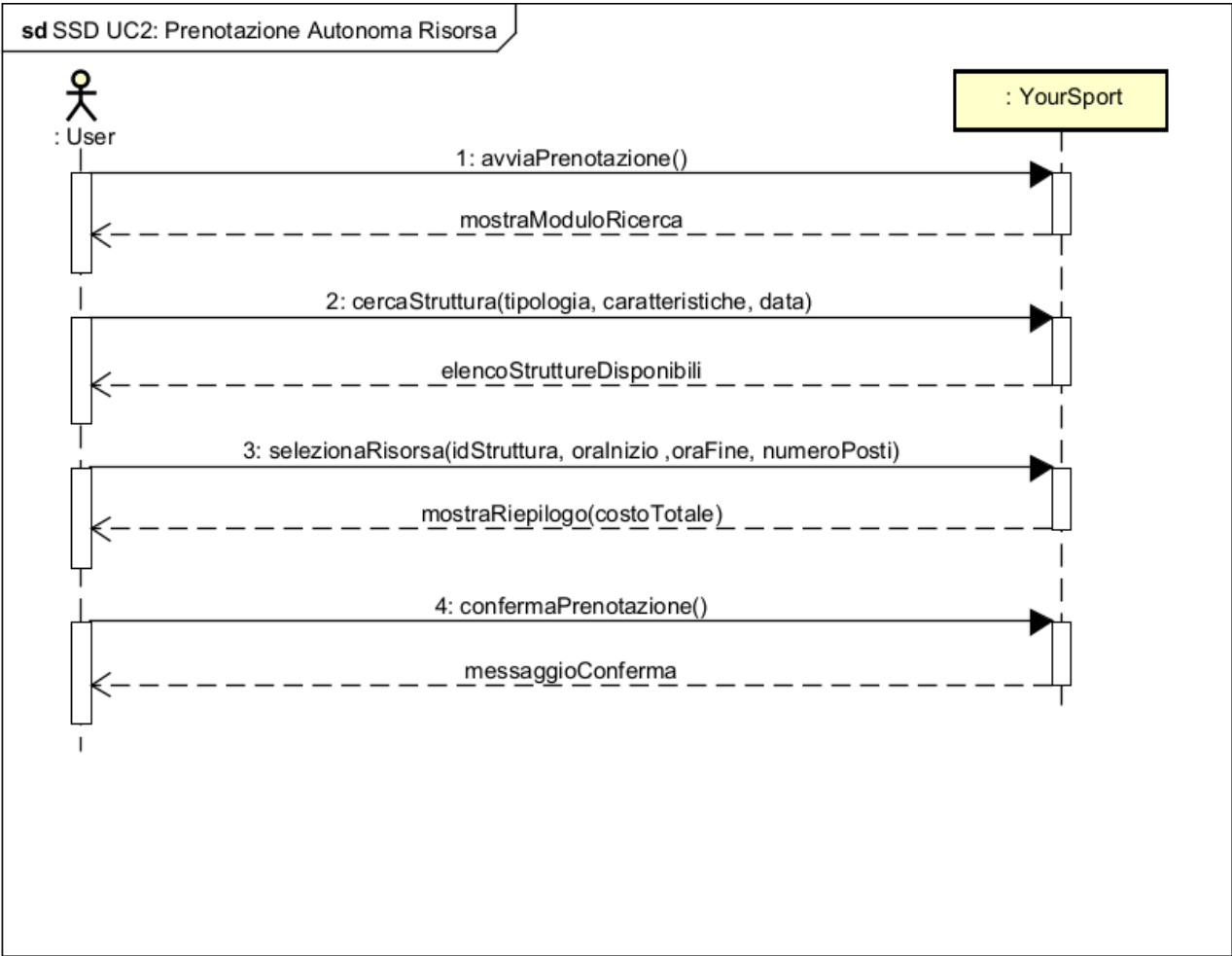


Diagramma di sequenza di sistema



Contratti delle operazioni

Operazioni	cercaStruttura(tipologia: String; caratteristiche: List<String>; data: Date)
Riferimenti	UC2: Prenotazione autonoma risorsa
Pre-condizioni	Lo Sportivo è autenticato nel sistema
Post-condizioni	Viene restituito un elenco di Strutture che soddisfano i criteri di ricerca

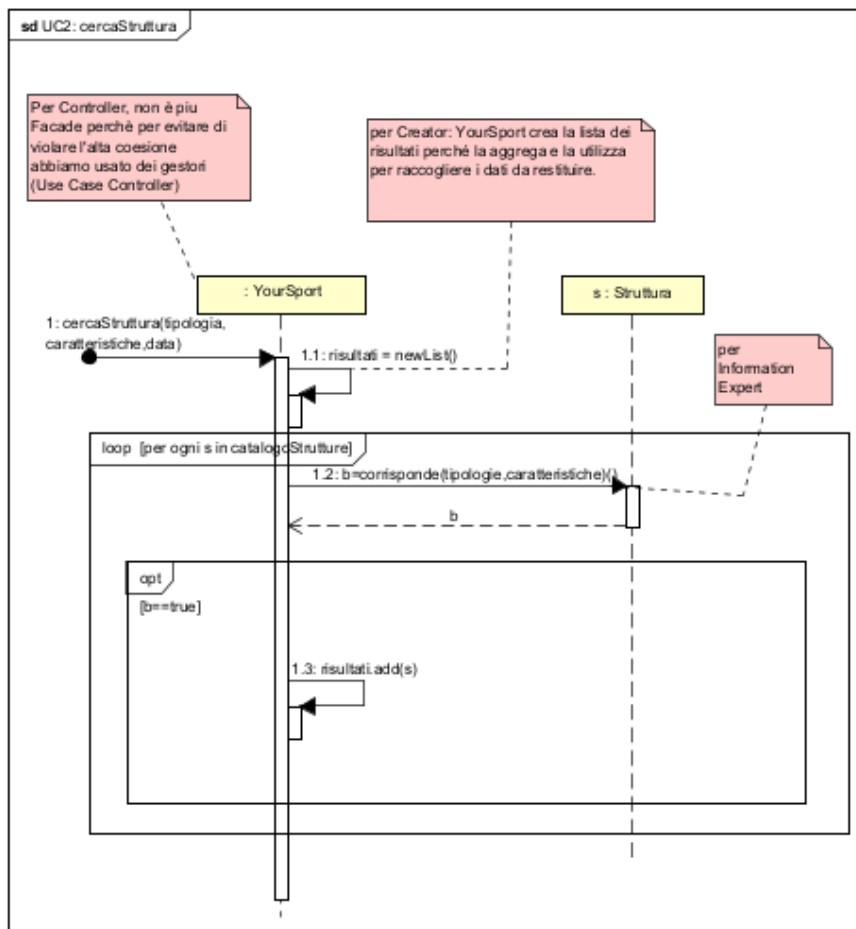
Operazioni	selezionaRisorsa(idStruttura: String; oraInizio: Time; oraFine: Time; numeroPosti: Int)
Riferimenti	UC2: Prenotazione autonoma risorsa
Pre-condizioni	È stata effettuata una ricerca e le strutture sono visibili

Post-condizioni	- È stata creata una nuova istanza p di Prenotazione - p è stata associata allo Sportivo corrente - p è stata associata alla Struttura indentificata da idStruttura - Gli attributi di p (oraInizio, oraFine, numeroPosti, data) sono stati impostati in base agli argomenti - L'attributo p.costoTotale è stato calcolato e impostato basandosi sulla tariffa della Struttura
-----------------	--

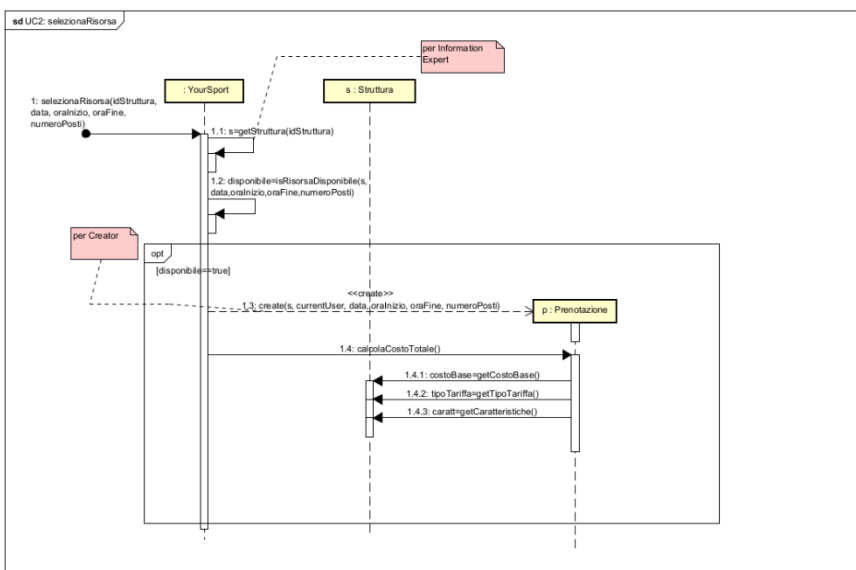
Operazioni	confermaPrenotazione()
Riferimenti	UC2: Prenotazione autonoma risorsa
Pre-condizioni	Esiste una prenotazione p in corso associata allo Sportivo
Post-condizioni	- Lo stato di p diventa "Confermata" - L'istanza p viene associata al registro archivioPrenotazioni dell'oggetto YourSport

Diagrammi di sequenza

-cercaStruttura



-selezionaRisorsa



-confermaPrenotazione

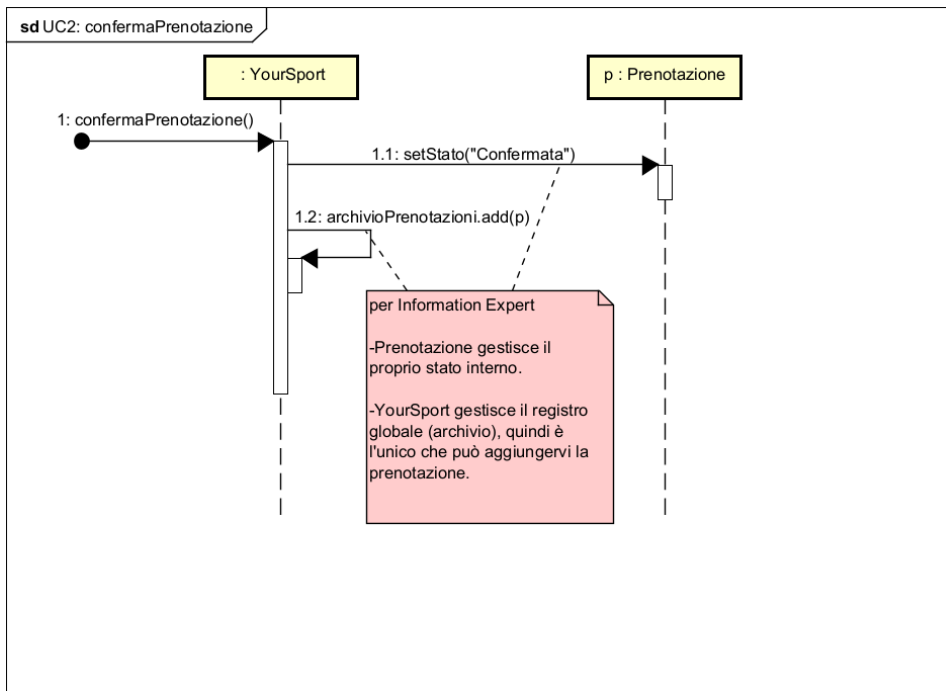
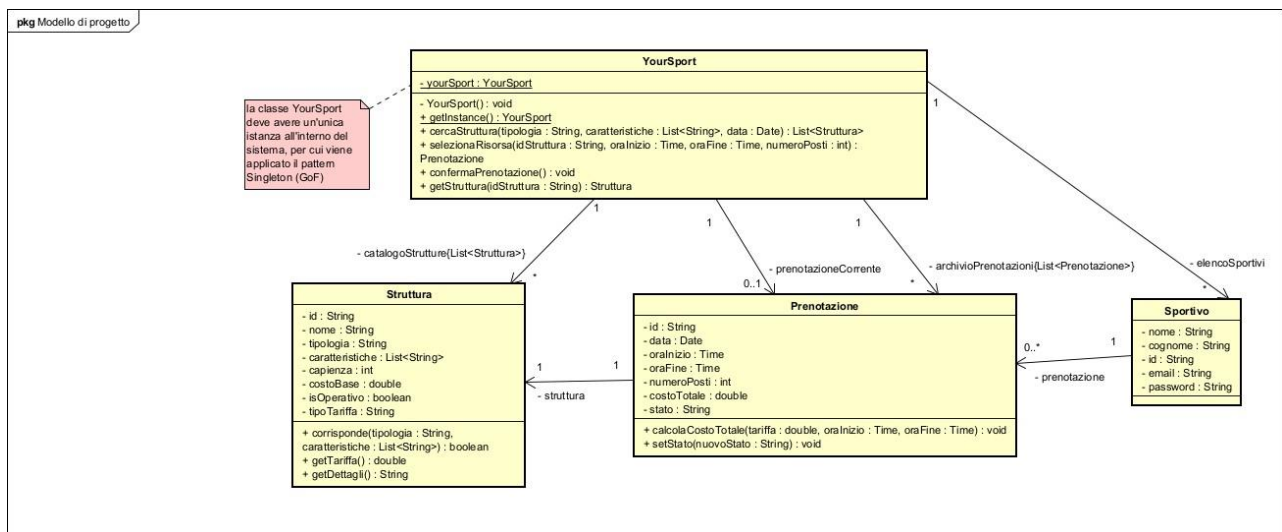


Diagramma delle classi di progetto



Testing

Per il caso d'uso UC2 abbiamo ritenuto opportuno fare i seguenti test

-Test 1: La Ricerca (Filtro Catalogo)

- **Descrizione:** Effettuiamo una ricerca generica nel sistema chiedendo di visualizzare solo i campi da "Tennis", senza specificare altri filtri.
- **Scopo:** Verificare che la funzione di ricerca sappia filtrare correttamente il catalogo delle strutture in base alla tipologia richiesta, escludendo quelle non pertinenti.

- **Realizzazione:** Il test interroga il sistema cercando la parola "Tennis". Successivamente, controlla che la lista dei risultati contenga esattamente un solo elemento e che questo sia effettivamente un campo da tennis (ignorando la piscina).

Test 2: Prenotazione Tennis (Logica Esclusiva e Conflitto)

- **Descrizione:** Simuliamo due persone che cercano di prenotare lo stesso identico campo da tennis per la stessa identica fascia oraria (dalle 10:00 alle 11:00).
- **Scopo:** Assicurarsi che il sistema prevenga l'overbooking per le strutture ad "uso esclusivo" (tariffa ORARIO). Se il campo è occupato, nessun altro deve poterlo prenotare, nemmeno per una sola persona.
- **Realizzazione:** Il test esegue una prima prenotazione con successo. Subito dopo, lancia una seconda richiesta per lo stesso campo e lo stesso orario. Il test verifica che il sistema blocchi la seconda richiesta restituendo un "null" (fallimento).

-Test 3: Prenotazione Piscina (Logica Cumulativa per Capienza)

- **Descrizione:** Simuliamo l'ingresso progressivo di diversi gruppi di persone in una struttura condivisa (la Piscina, che ha 20 posti totali) nello stesso orario.
- **Scopo:** Verificare che il sistema sia in grado di sommare i posti occupati (tariffa PERSONA) e che blocchi le nuove prenotazioni solo nel momento in cui la capienza massima matematica viene superata.
- **Realizzazione:** Il test prenota prima 15 posti (lasciandone 5 liberi), poi altri 4 posti (lasciandone 1 libero). Entrambe le operazioni vanno a buon fine. Infine, tenta di prenotare 2 posti: il test verifica che il sistema rifiuti quest'ultima operazione, poiché si supererebbe il limite massimo ($15 + 2 > 20$).

-Test 4: Prenotazione Piscina (Indipendenza degli Orari)

- **Descrizione:** Testiamo cosa succede se riempiamo completamente la piscina al mattino e poi cerchiamo di prenotarla per il pomeriggio dello stesso giorno.
- **Scopo:** Dimostrare che il sistema analizza la disponibilità in modo "chirurgico" sulle singole fasce orarie. Una struttura "Sold Out" al mattino non deve risultare chiusa per il resto della giornata.
- **Realizzazione:** Il test prenota tutti e 20 i posti disponibili in piscina dalle 10:00 alle 11:00. Successivamente, tenta di prenotare 5 posti dalle 15:00 alle 16:00. Il test verifica che questa seconda prenotazione venga accettata con successo.

-Test 5: Filtro per Caratteristiche

- **Descrizione:** Cerchiamo una struttura basandoci solo su un servizio offerto (ad esempio, la presenza della "Doccia"), senza specificare lo sport.
- **Scopo:** Testare l'efficacia dei filtri avanzati. Il sistema deve scansionare le caratteristiche di ogni struttura e restituire solo quelle che possiedono il requisito richiesto.
- **Realizzazione:** Il test passa una lista contenente la caratteristica "Doccia" e lascia in bianco il campo dello sport. Poi analizza i risultati verificando che il sistema abbia trovato la Piscina (che ha la doccia) e abbia scartato il Campo da Tennis.

-Test 6: Ricerca a Vuoto (Nessun Risultato)

- **Descrizione:** Chiediamo al sistema di cercare uno sport che sappiamo non esistere nel nostro catalogo (ad esempio, "Golf").
- **Scopo:** Verificare la stabilità del programma in caso di ricerche infruttuose. Il sistema deve gestire la situazione in modo elegante, senza andare in crash o restituire errori anomali (come un valore *null* non gestito).
- **Realizzazione:** Il test lancia la ricerca per la parola "Golf". Verifica poi due cose fondamentali: primo, che la risposta del sistema esista (non sia un errore grave/null); secondo, che la lista dei risultati sia perfettamente vuota, come ci si aspetta.

Iterazione 2

Nella seconda iterazione lo sviluppo si è concentrato sul fornire all'Amministratore gli strumenti necessari per la gestione logistica ed economica dell'impianto. In particolare, è stato implementato il **Caso d'Uso 3 (Gestione dei costi di prenotazione)**, che regola la modifica dinamica delle politiche di prezzo delle singole strutture. Inoltre, in questa fase è stato raffinato il Modello di Dominio.

I pattern architetturali utilizzati sono i seguenti:

- **Facade Controller (GRASP):** La classe YourSport riceve l'input amministrativo e gestisce le logiche di controllo.
- **Information Expert (GRASP):** Nel recupero dei dettagli, il sistema delega la formulazione del riepilogo direttamente all'oggetto Struttura. Allo stesso modo, durante la fase di aggiornamento, il Controller non forza la manipolazione diretta delle variabili, ma invoca in modo sicuro i metodi *setter* interni alla classe esperta Struttura

UC3: Gestione dei costi di prenotazione

Nome caso d'uso	UC3: Gestione dei costi di prenotazione
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Admin
Parti interessate e interessi	• Admin: vuole aggiornare i listini prezzi delle strutture in modo flessibile
Pre-condizioni	L'admin è autenticato con i privilegi amministrativi
Garanzie di successo	I costi delle strutture vengono aggiornati e sono immediatamente visibili per le nuove prenotazioni

Scenario principale di successo	1. L'Admin seleziona l'impianto o la specifica struttura da modificare. 2. Il Sistema mostra i parametri attuali. 3. L'Admin seleziona il tipo di tariffazione: "Affitto intera struttura" o "Ingresso singolo" 4. L'Admin imposta il prezzo base (orario o per persona) 5. L'Admin conferma le modifiche. 6. Il Sistema salva le nuove tariffe e le applica alle future prenotazioni. <i>I passi da 1 a 5 vengono ripetuti finché serve.</i> 7. L'admin indica di aver finito.
Estensioni	*a. In qualsiasi momento il sistema fallisce e ha un arresto improvviso. 1. L'admin riavvia il software e ripristina lo stato precedente del sistema. 2. Il sistema ripristina lo stato. 3a. L'Admin inserisce un valore non valido (es. negativo o lettere). 1. Il Sistema mostra un messaggio di errore. 2. L'Admin ripete il passo 3 inserendo i valori corretti

Modello di dominio:

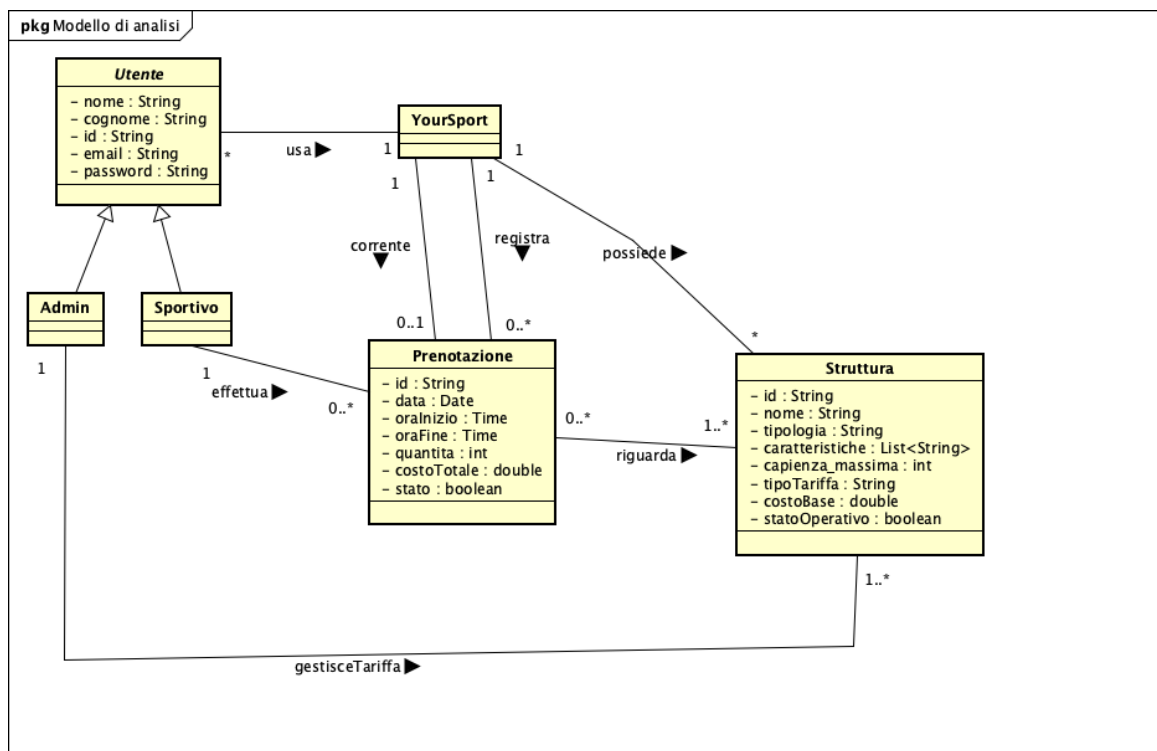
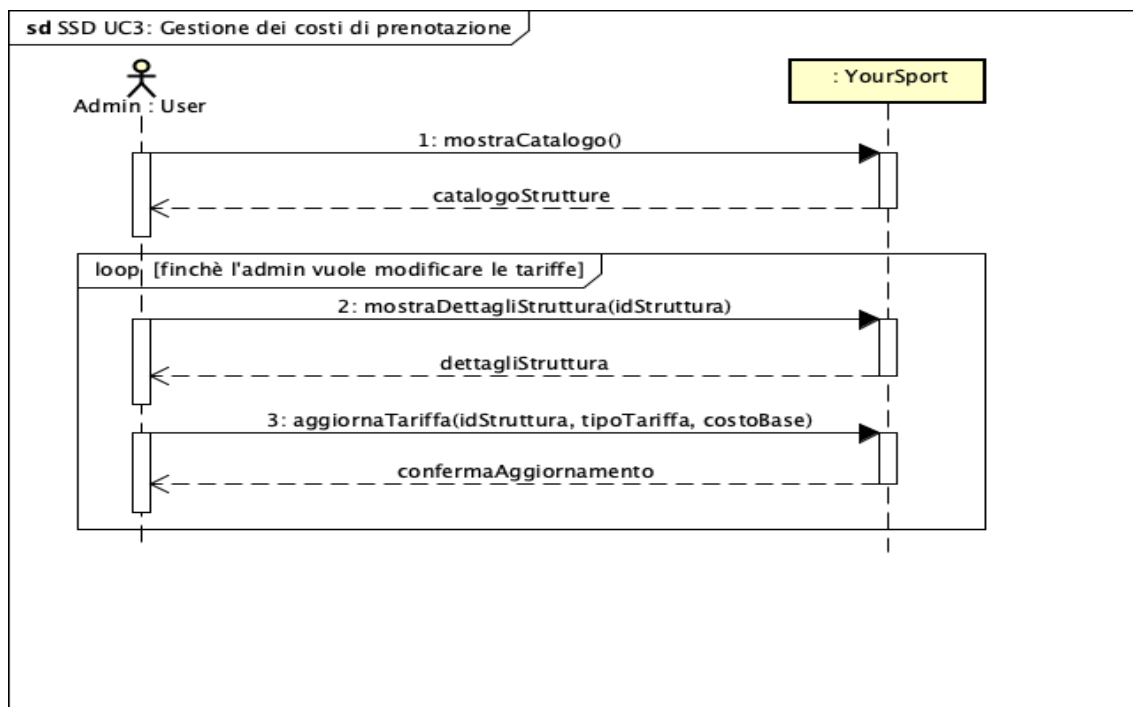


Diagramma di sequenza di sistema:



Contratti delle operazioni:

UC3

Operazioni	mostraCatalogo()
Riferimenti	UC3: Gestione dei costi di prenotazione
Pre-condizioni	L'utente corrente è autenticato nel sistema come Admin
Post-condizioni	Viene restituito l'elenco globale catalogoStrutture contenente tutte le istanze di Struttura

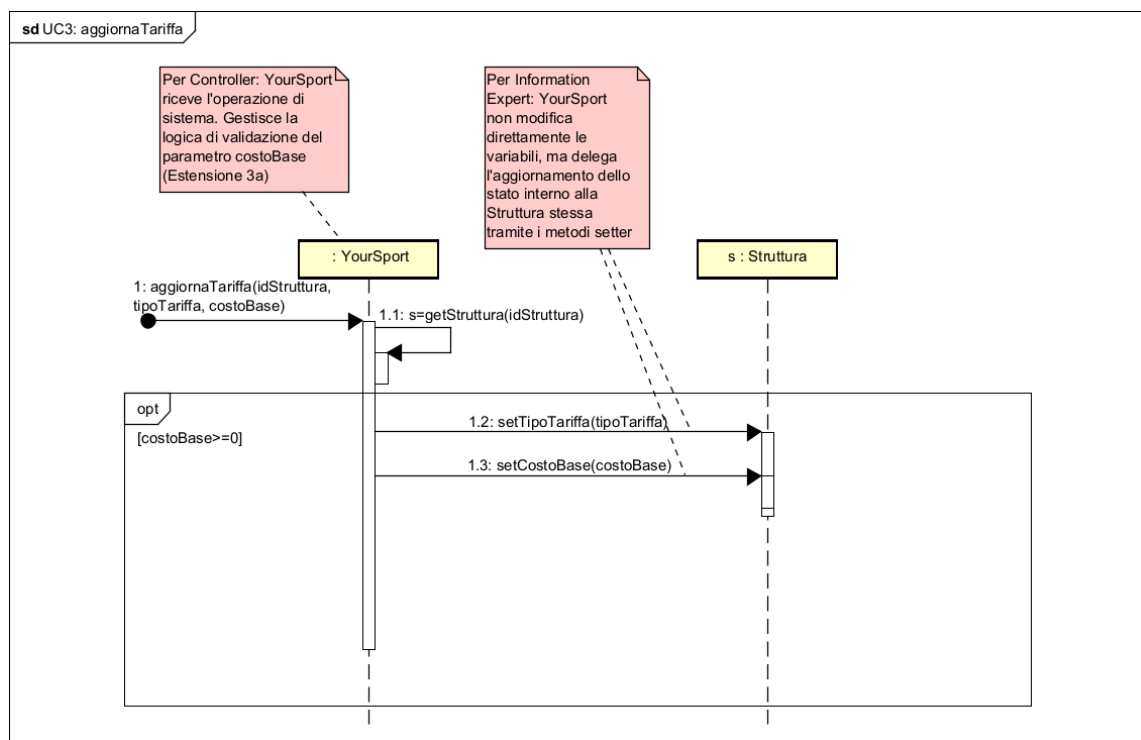
Operazioni	mostraDettagliStruttura(idStruttura: String)
Riferimenti	UC3: Gestione dei costi di prenotazione
Pre-condizioni	Esiste un'istanza s di Struttura il cui identificativo corrisponde a idStruttura
Post-condizioni	Vengono restituiti i dettagli dell'istanza s (inclusi i valori attuali degli attributi tipoTariffa e costoBase)

Operazioni	aggiornaTariffa(idStruttura: String, tipoTariffa: String, costoBase: double)
Riferimenti	UC3: Gestione dei costi di prenotazione
Pre-condizioni	- L'utente corrente è un Admin. - Esiste un'istanza s di Struttura associata a idStruttura. - Il valore di costoBase inserito è un numero valido e positivo

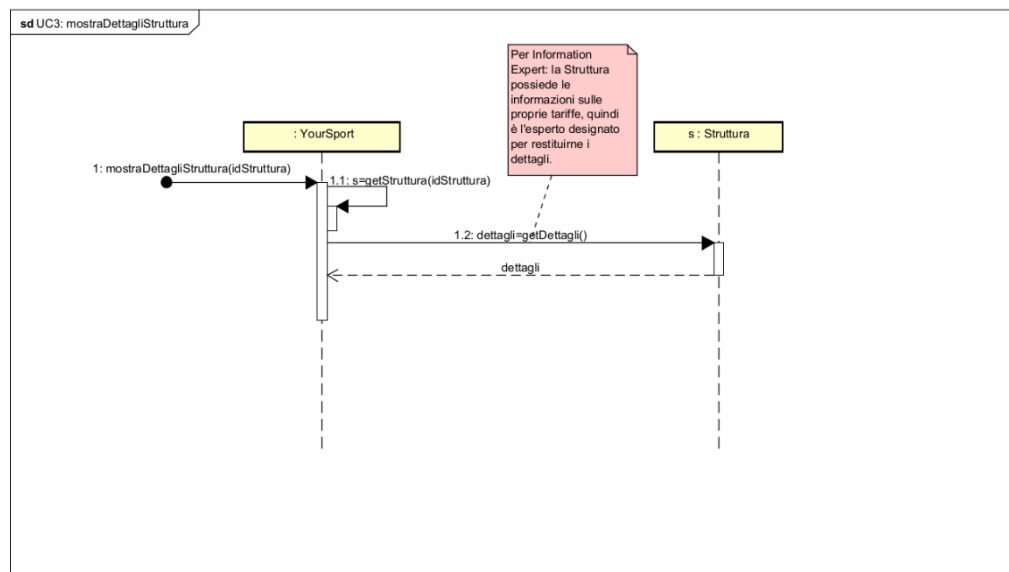
	(gestione estensione 3a del caso d'uso).
Post-condizioni	L'attributo tipoTariffa dell'istanza s è stato modificato e impostato al valore del parametro tipoTariffa.

Diagrammi di sequenza:

-aggiornaTariffa



-mostraDettagliStruttura



Testing:

Test 1: Visualizzazione Catalogo (Mostra Strutture)

Descrizione: Simuliamo l'amministratore che accede al pannello di gestione e richiede di vedere la lista di tutte le strutture presenti nel sistema per decidere quale modificare.

Scopo: Assicurarsi che il sistema (tramite il Controller) restituisca correttamente la collezione delle strutture inizializzate in memoria, verificando che il collegamento al database (in questo caso la lista in memoria) sia attivo.

Realizzazione: Il test chiama il metodo `mostraCatalogo()` e verifica tramite asserzioni (`assertNotNull`, `assertTrue`) che la lista ottenuta non sia vuota o nulla, ma contenga almeno i dati di default (es. il Campo A e la Piscina).

Test 2: Dettagli Struttura (Pattern Information Expert)

Descrizione: Simuliamo l'amministratore che seleziona una struttura specifica (es. "S1", Campo da Tennis) digitandone l'ID per visualizzarne la scheda tecnica e le tariffe attuali.

Scopo: Verificare che il Controller deleghi correttamente la creazione della stringa di riepilogo alla classe Struttura, rispettando il pattern GRASP Information Expert modellato nel Diagramma di Sequenza.

Realizzazione: Il test passa un ID valido al metodo mostraDettagliStruttura(). Verifica poi che la stringa restituita contenga i dati corretti (come il nome "Campo A" e il prezzo "20.0") e non messaggi d'errore.

Test 3: Dettagli Struttura Inesistente (Gestione Errore)

Descrizione: Simuliamo l'errore umano in cui l'amministratore digita un ID struttura che non esiste nel sistema (es. "S999").

Scopo: Garantire che il sistema non vada in crash (sollevando una NullPointerException) quando cerca una struttura mancante, ma gestisca l'anomalia in modo controllato.

Realizzazione: Il test invoca il metodo passando un ID fittizio e verifica che la stringa restituita corrisponda all'esatto messaggio di errore previsto dal sistema ("Errore: Struttura inesistente").

Test 4: Aggiornamento Tariffa (Flusso Principale/Happy Path)

Descrizione: Simuliamo l'amministratore che decide di cambiare le regole commerciali di una struttura, modificandone con successo sia il tipo di tariffa (es. passandola a PERSONA) sia il costo base (es. alzandolo a 25€).

Scopo: Confermare che l'operazione di sistema aggiornaTariffa modifichi correttamente e permanentemente gli attributi della struttura bersaglio, rispettando le post-condizioni pattuite nei Contratti delle Operazioni.

Realizzazione: Il test legge i dati attuali, chiama il metodo di aggiornamento con nuovi valori validi e infine rilegge la struttura, verificando matematicamente che il prezzo e il tipo siano stati effettivamente sovrascritti.

Test 5: Inserimento Costo Negativo (Estensione 3a)

Descrizione: Simuliamo l'amministratore che digita accidentalmente un prezzo inferiore a zero (es. -5€) durante la fase di aggiornamento della tariffa.

Scopo: Verificare che il sistema applichi rigorosamente il blocco OPT modellato nel Diagramma di Sequenza ([costoBase >= 0]), impedendo di salvare nel sistema prezzi impossibili.

Realizzazione: Il test tenta di impostare un valore negativo. Subito dopo, rilegge l'oggetto Struttura e verifica che il prezzo sia rimasto identico a quello precedente al tentativo, confermando che l'aggiornamento è stato bloccato.

Test 6: Inserimento Tipo Tariffa Non Valido (Programmazione Difensiva)

Descrizione: Simuliamo l'amministratore che inserisce una parola non riconosciuta dal sistema (es. "GIORNATA" o "A ORE") al posto dei comandi standard "ORARIO" o "PERSONA".

Scopo: Proteggere l'integrità dell'algoritmo di disponibilità dell'UC2. Se il sistema accettasse stringhe casuali, le future prenotazioni si buggherebbero perché il sistema non saprebbe come calcolare la capienza.

Realizzazione: Il test invia una stringa "spazzatura" come tipo di tariffa. Verifica poi che l'operazione venga scartata istantaneamente e che né il tipo né il prezzo della struttura subiscano alterazioni, confermando la robustezza del codice.

Iterazione 3

Durante l'Iterazione 3, lo sviluppo si è concentrato sull'interazione diretta tra l'utente finale e il sistema, affrontando il **Caso d'Uso 1 (Registrazione alla piattaforma)** e il **Caso d'Uso 4 (Gestione delle proprie prenotazioni)**

I pattern utilizzati sono:

- **Creator (GRASP):** Nell'UC1, la classe YourSport assume la responsabilità di istanziare il nuovo oggetto Sportivo poiché, al termine della creazione, dovrà aggregarlo e memorizzarlo all'interno del proprio elenco.
- **Information Expert (GRASP):** Sempre nell'UC1, YourSport funge da esperto per la verifica dell'esistenza dell'email, essendo l'unica classe a possedere fisicamente l'informazione necessaria (elencoSportivi). Nell'UC4, YourSport viene interrogato dal gestore per recuperare lo specifico Sportivo.
- **Pure Fabrication/Controller (GRASP):** Questa è l'innovazione architetturale principale dell'iterazione. Per gestire le operazioni dell'UC4 (mostrare e annullare le prenotazioni), abbiamo introdotto una nuova classe di controllo dedicata chiamata GestorePrenotazioni. Questa scelta è stata fatta per mantenere alta la coesione e prevenire il sovraccarico della classe principale YourSport.

UC1: Registrazione alla piattaforma

Nome caso d'uso	UC1: Registrazione alla piattaforma
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Visitatore
Parti interessate e interessi	Visitatore: vuole inserire i propri dati personali per creare un account e poter diventare uno Sportivo registrato.
Pre-condizioni	Il visitatore ha accesso all'applicazione/sito web.
Garanzie di successo	I dati del visitatore sono salvati nel sistema e il nuovo profilo Sportivo è creato correttamente.

Scenario principale di successo	1. Il Visitatore accede alla sezione di registrazione. 2. Il Sistema richiede i dati anagrafici e le credenziali (Nome, Cognome, Email, Password). 3. Il Visitatore inserisce i dati richiesti e conferma la registrazione. 4. Il Sistema verifica che l'indirizzo email inserito non sia già presente nel database. 5. Il Sistema registra il nuovo profilo, lo aggiunge all'elenco degli sportivi e mostra un messaggio di successo.
---------------------------------	---

Estensioni

***a.** In qualsiasi momento il sistema fallisce e ha un arresto improvviso.

1. L'admin riavvia il software e ripristina lo stato precedente del sistema.

2. Il sistema ripristina lo stato.

4a. L'email inserita risulta già registrata nel sistema.

1. Il Sistema mostra un messaggio di errore ("Email già in uso").

2. Il Sistema chiede al Visitatore di inserire un'email diversa o di annullare l'operazione.

Modello di dominio:

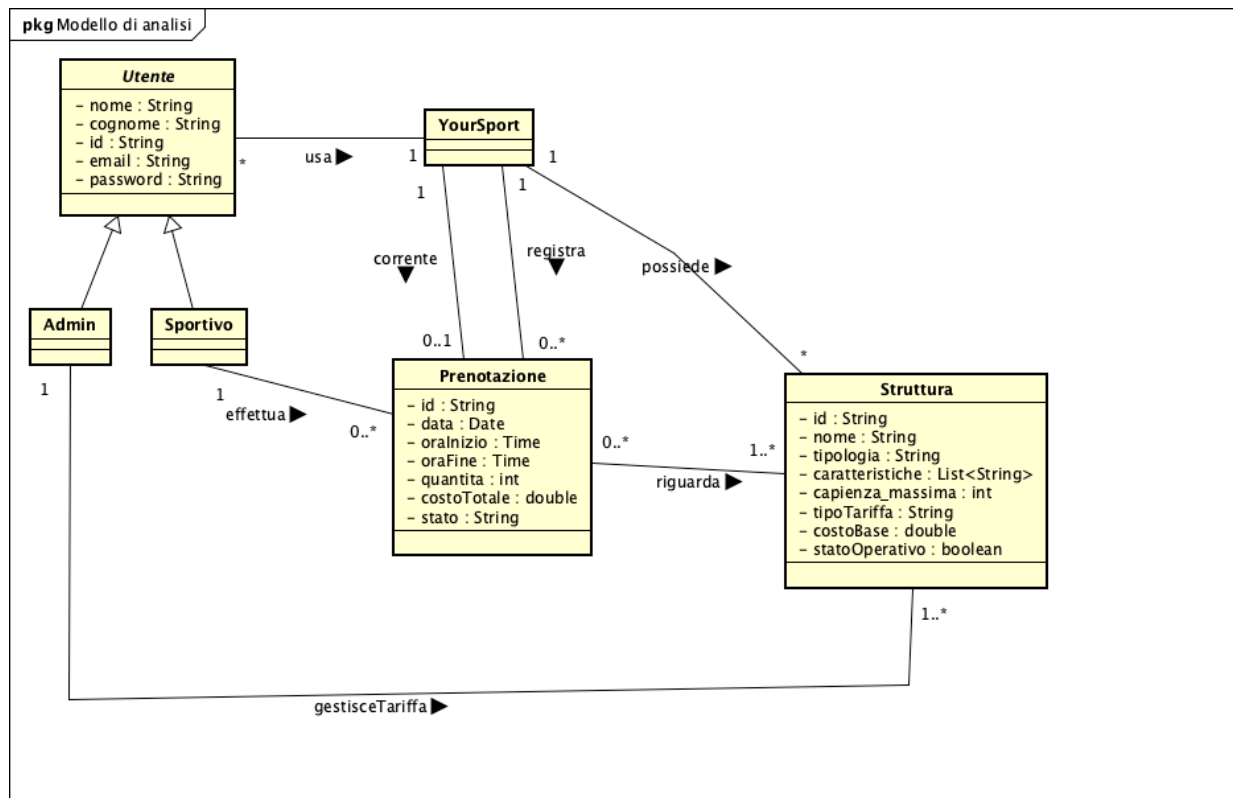
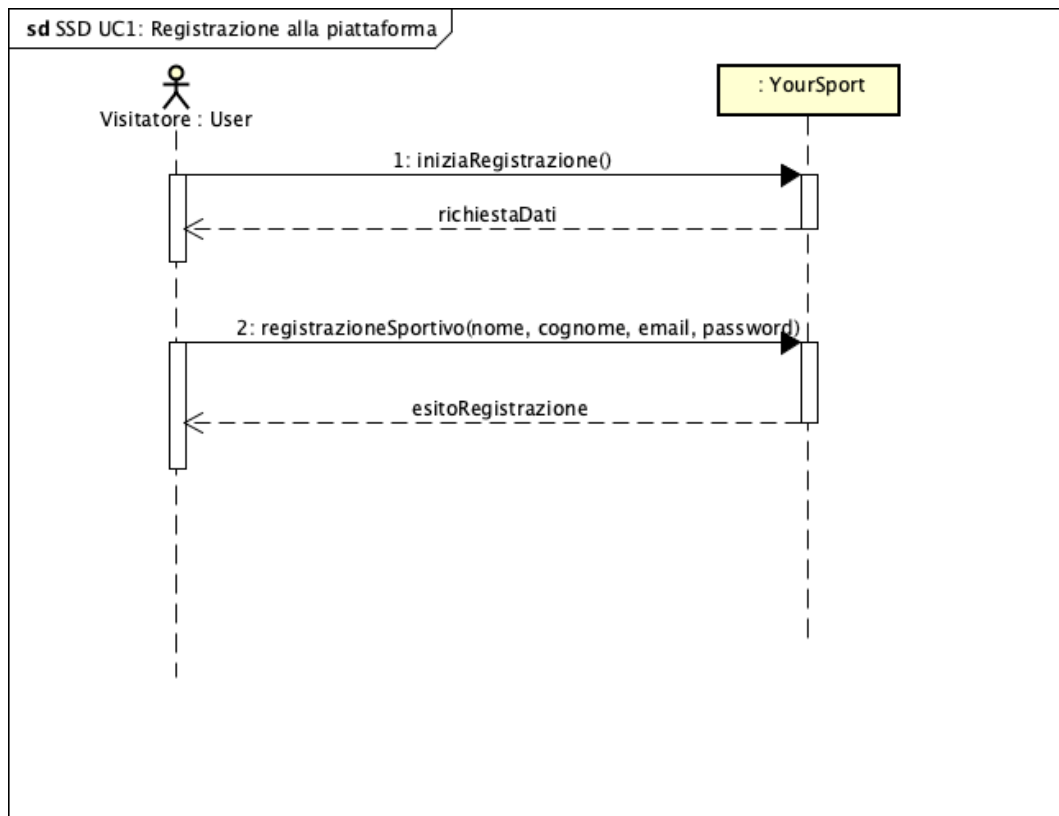


Diagramma di sequenza di sistema:



Contratti delle operazioni:

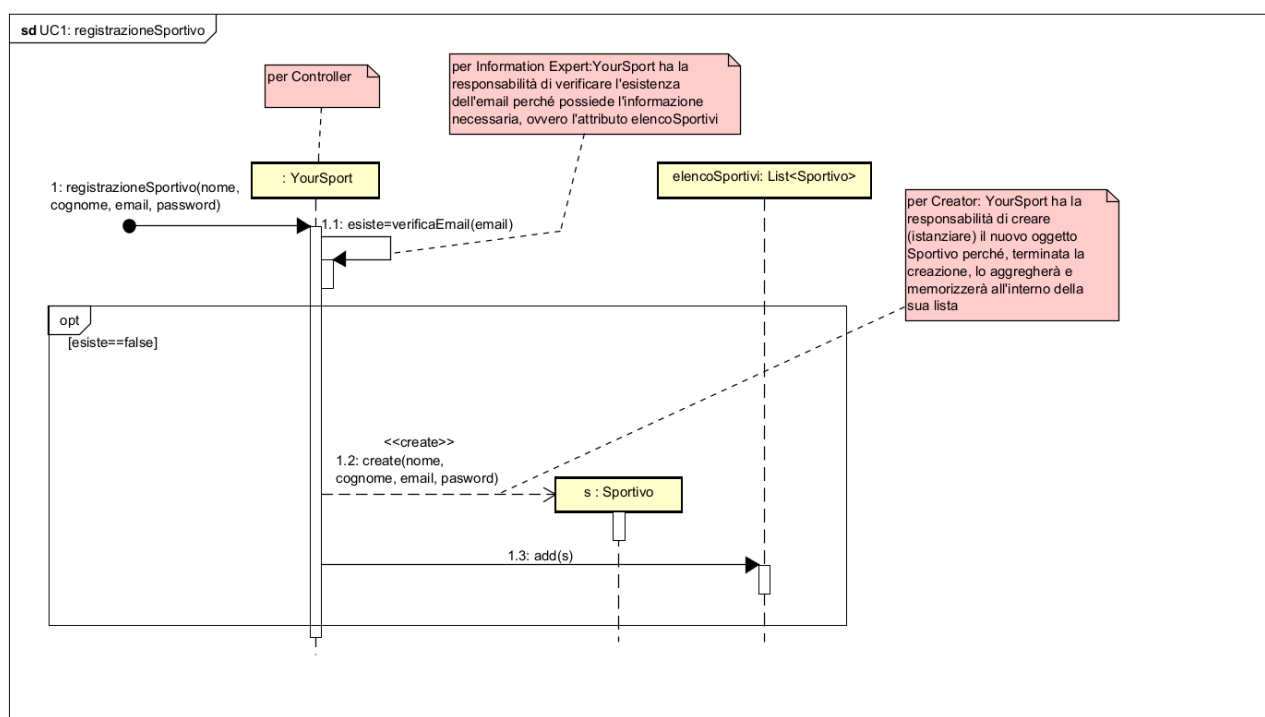
-UC1

Operazioni	registrazioneSportivo(nome: String, cognome: String, email: String, password: String)
Riferimenti	UC1: Registrazione alla piattaforma
Pre-condizioni	La stringa email non è presente in nessuna istanza di Sportivo già registrata nel sistema.

Post-condizioni	- È stata creata una nuova istanza s della classe Sportivo (<i>Creazione di istanza</i>). - Gli attributi nome, cognome, email e password dell'istanza s sono stati inizializzati con i valori dei parametri corrispondenti (<i>Modifica di attributi</i>). - L'istanza s è stata associata all'oggetto YourSport (inserita nel catalogo/elenco degli sportivi del sistema) (<i>Formazione di associazione</i>).
-----------------	---

Diagrammi di sequenza:

-registrazioneSportivo



Testing:

Test 1: Registrazione Nuovo Utente con Successo (Happy Path)

- **Descrizione:** Simuliamo un nuovo utente non ancora presente nel sistema che inserisce i propri dati (nome, cognome, email libera e password) per creare un account.

- **Scopo:** Verificare che il sistema elabori correttamente il flusso principale dell'UC1, istanziando il nuovo oggetto Sportivo (Pattern Creator) e aggregandolo alla collezione degli utenti del sistema.
- **Realizzazione:** Il test chiama il metodo registrazioneSportivo() passando dati del tutto nuovi. Si verifica tramite asserzione (assertEquals) che il sistema restituisca la stringa "utente registrato". Come step finale, si esegue una verifica strutturale (tramite assertTrue invocando il metodo login) per accertarsi che il nuovo utente esista effettivamente in memoria e possa autenticarsi.

Test 2: Registrazione Bloccata per Email Duplicata (Flusso Alternativo)

- **Descrizione:** Simuliamo un utente che tenta di registrarsi utilizzando un indirizzo email già associato a un account esistente nel sistema (nel nostro caso, la mail di default "mario@email.it").
- **Scopo:** Assicurarsi che il sistema rispetti la pre-condizione e il blocco OPT del Diagramma di Sequenza, impedendo la creazione di account cloni e mantenendo la coerenza dei dati.
- **Realizzazione:** Il test chiama registrazioneSportivo() passando un indirizzo email già noto al sistema. Tramite assertEquals si verifica che il sistema intercetti la collisione (grazie al metodo verificaEmail) e restituisca esattamente il messaggio d'errore previsto: "email già in uso", interrompendo la creazione dell'oggetto.

UC4: Gestione delle proprie prenotazioni

Nome caso d'uso	UC4: Gestione delle proprie prenotazioni
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Sportivo (Utente registrato)
Parti interessate e interessi	Sportivo: vuole tenere traccia delle proprie attività e cancellare impegni futuri in caso di imprevisti.
Pre-condizioni	Lo sportivo ha effettuato il login al sistema.
Garanzie di successo	Lo stato della prenotazione viene modificato e, in caso di annullamento, la struttura torna nuovamente disponibile per altri sportivi.
Scenario principale di successo	1. Lo Sportivo accede alla propria area personale. 2. Il Sistema recupera e mostra l'elenco e lo stato delle prenotazioni (sia future che passate) associate a quello specifico sportivo. 3. Lo Sportivo seleziona una prenotazione futura e richiede di annullarla. 4. Il Sistema richiede un'ulteriore conferma per procedere con l'annullamento. 5. Lo Sportivo conferma l'operazione. 6. Il Sistema aggiorna lo stato della prenotazione in "Annullata" e rende la struttura nuovamente disponibile per la data e la fascia oraria corrispondente. 7. Il Sistema mostra un messaggio di successo allo Sportivo.

Estensioni

***a.** In qualsiasi momento il sistema fallisce e ha un arresto improvviso.

1. L'admin riavvia il software e ripristina lo stato precedente del sistema.

2. Il sistema ripristina lo stato.

3a. Lo Sportivo tenta di annullare una prenotazione passata o attualmente in corso.

1. Il Sistema segnala che l'operazione non è consentita per eventi già iniziati o trascorsi.

2. Lo Sportivo torna alla visualizzazione dell'elenco.

5a. Lo Sportivo rifiuta la conferma di annullamento.

1. Il Sistema interrompe l'operazione, mantenendo la prenotazione attiva e confermata.

2. Lo Sportivo torna alla visualizzazione dell'elenco.

Modello di dominio:

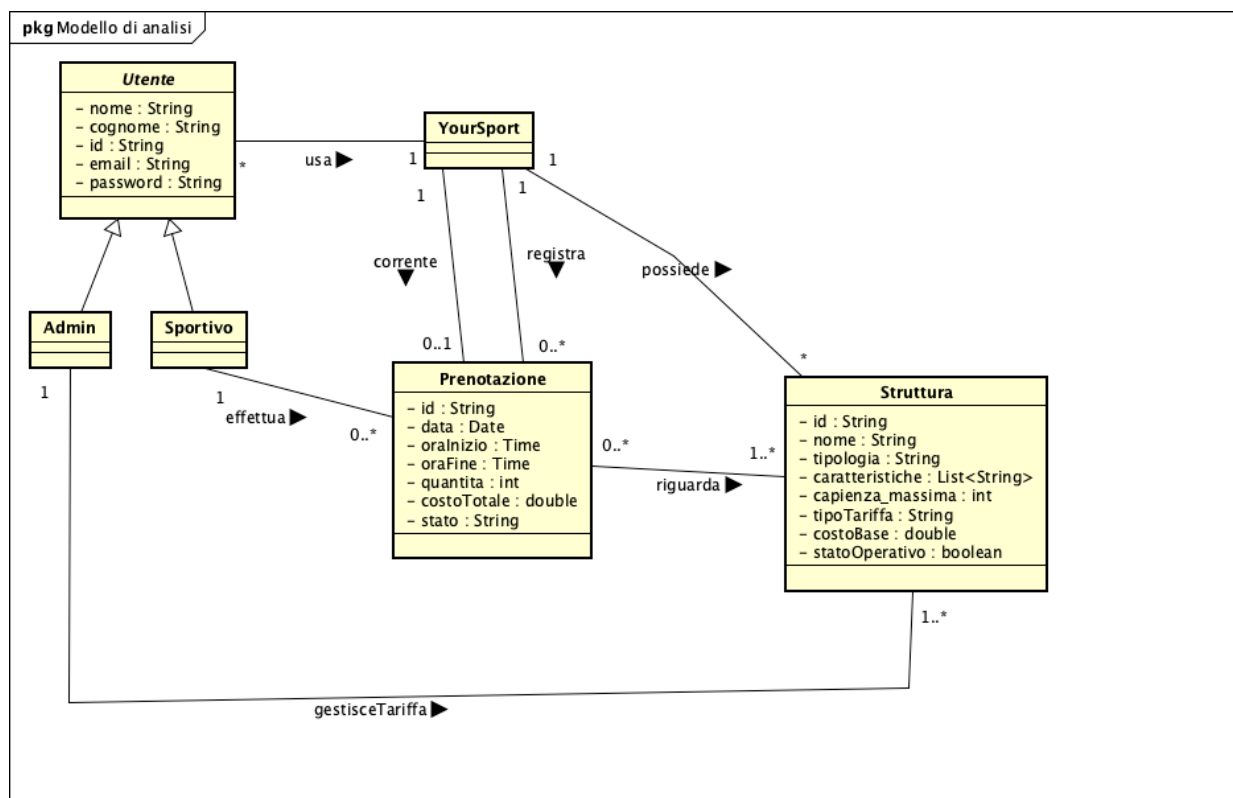
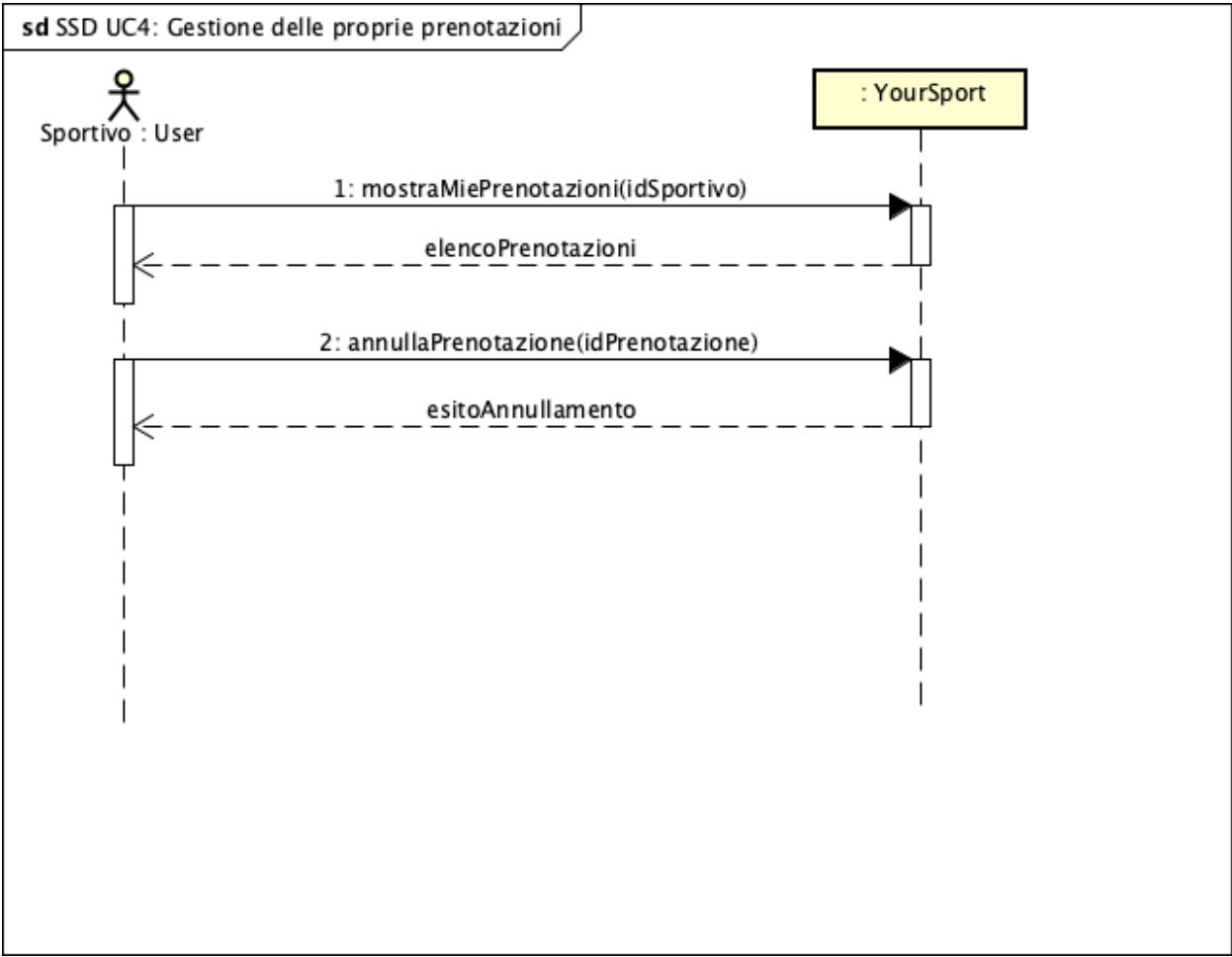


Diagramma di sequenza di sistema:



Contratti delle operazioni:

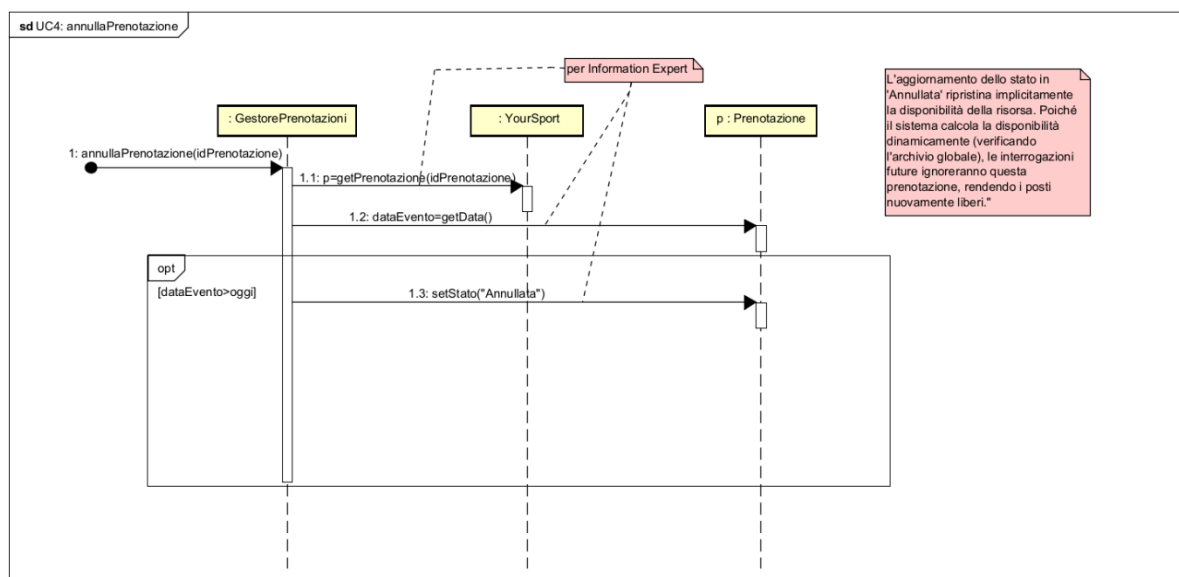
-UC4

Operazioni	annullaPrenotazione(idPrenotazione: String)
Riferimenti	UC4: Gestione delle proprie prenotazioni
Pre-condizioni	La prenotazione con idPrenotazione esiste nel sistema e la sua data/ora di inizio è successiva al momento attuale (è futura).

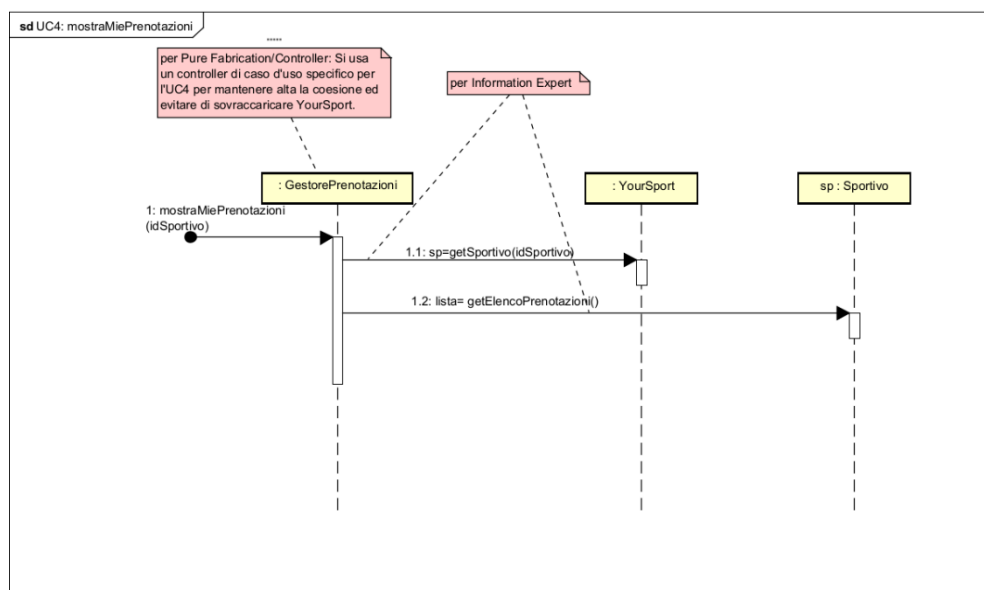
Post-condizioni	- L'attributo stato dell'istanza Prenotazione corrispondente è stato modificato in "Annullata" -La disponibilità della risorsa/struttura associata a quella prenotazione è stata ripristinata per quella data e fascia oraria
-----------------	---

Diagrammi di sequenza:

-annullaPrenotazione



-mostraMiePrenotazioni



Testing:

Test 1: Visualizzazione delle Proprie Prenotazioni

- **Descrizione:** Simuliamo l'utente loggato che accede alla propria area personale e chiede al sistema di visualizzare l'elenco delle prenotazioni da lui effettuate in precedenza.
- **Scopo:** Verificare che il nuovo controller (Pure Fabrication) `GestorePrenotazioni` comunichi correttamente con il Singleton `YourSport` per recuperare l'oggetto `Sportivo` corretto e ne restituisca la lista privata delle prenotazioni.
- **Realizzazione:** Il test prevede una fase di *setup* in cui il sistema crea una prenotazione valida per l'utente. Successivamente, si chiama il metodo `mostraMiePrenotazioni(id)`. Si utilizzano asserzioni (`assertNotNull`, `assertFalse`) per garantire che la lista ritornata non sia nulla o vuota, e un `assertEquals` finale per confermare che l'ID della prenotazione dentro la lista coincida esattamente con quello appena generato.

Test 2: Annullamento di una Prenotazione Futura (Happy Path)

- **Descrizione:** Simuliamo l'utente che decide di cancellare una sua prenotazione relativa a una data futura, fornendo al sistema il codice ID corretto.
- **Scopo:** Controllare il rispetto della guardia [`dataEvento > oggi`] nel blocco OPT dell'UC4, verificando che il sistema accetti l'annullamento e applichi la post-condizione, ovvero l'aggiornamento dello stato dell'oggetto.
- **Realizzazione:** Viene creata una prenotazione fittizia impostata per due giorni nel futuro. Si invoca il metodo `annullaPrenotazione()` passandone l'ID. Tramite `assertEquals` si verificano due cose fondamentali: che il messaggio di ritorno sia "prenotazione annullata" e, soprattutto, che l'attributo stato dell'oggetto `Prenotazione` in memoria sia effettivamente mutato in "Annullata".

Test 3: Tentativo di Annullamento con ID Inesistente (Flusso Alternativo)

- **Descrizione:** Simuliamo l'utente che, per errore o per manipolazione, inserisce nel sistema un codice ID di prenotazione non valido o non registrato nell'archivio.
- **Scopo:** Verificare la robustezza dell'implementazione e la corretta gestione degli input errati, assicurandosi che il sistema non vada in crash (es. `NullPointerException`) ma risponda in modo controllato.
- **Realizzazione:** Il test invoca il metodo `annullaPrenotazione("PRN-FALSO")`. Utilizzando un `assertEquals`, si verifica che il controller intercetti la mancanza dell'oggetto e restituisca immediatamente la stringa di cortesia: "errore: prenotazione inesistente"

Iterazione 4

Durante l'Iterazione 4, lo sviluppo si è esteso alle funzionalità di manutenzione dell'impianto e gestione delle emergenze. Il lavoro si è concentrato sull'implementazione del **Caso d'Uso 5 (Segnalazione problemi struttura)**, del **Caso d'Uso 6 (Gestione segnalazioni)** e del **Caso d'Uso 7 (Sospensione e Riattivazione Struttura)**.

Pattern Utilizzati:

- **Pure Fabrication (Controller):** Per prevenire il sovraccarico di responsabilità sulla classe principale YourSport, in questa iterazione abbiamo introdotto due nuovi controller dedicati: GestoreSegnalazioni (UC6) e GestoreStrutture (UC7). Questo garantisce che l'architettura mantenga un'alta coesione.
- **Creator (GRASP):** Il pattern è stato applicato in più scenari: per l'istanziamento di un nuovo oggetto Segnalazione generato dallo sportivo (UC5) e per la creazione automatizzata dei messaggi di Notifica durante l'annullamento a cascata (UC7).
- **Information Expert (GRASP):** Il recupero e la manipolazione delle informazioni sono stati delegati alle classi che possiedono i dati necessari, come YourSport che viene interrogato dai gestori per recuperare le istanze corrette di Strutture e Segnalazioni.

UC5: Segnalazione problemi struttura

Nome caso d'uso	UC5: Segnalazione problemi struttura
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Sportivo (Utente registrato)
Parti interessate e interessi	Sportivo: desidera segnalare malfunzionamenti o danni per garantire la qualità del servizio e la sicurezza durante l'attività.
Pre-condizioni	Lo sportivo ha effettuato il login al sistema.
Garanzie di successo	Una nuova istanza di "Segnalazione" viene creata, associata alla struttura e resa visibile all'amministratore.

Scenario principale di successo	1. Lo Sportivo accede alla sezione "Segnala Guasto". 2. Il Sistema richiede di selezionare la struttura coinvolta (es. Campo Tennis A). 3. Lo Sportivo seleziona la struttura e inserisce una descrizione dettagliata del problema (es. "Rete rotta"). 4. Lo Sportivo conferma l'invio della segnalazione. 5. Il Sistema registra la segnalazione, le assegna uno stato iniziale ("Aperta"), acquisisce la data corrente e la associa allo sportivo e alla struttura. 6. Il Sistema conferma allo Sportivo l'avvenuto invio.
---------------------------------	--

Estensioni

***a.** In qualsiasi momento il sistema fallisce.

1. L'admin riavvia il software e ripristina lo stato.

3a. Lo Sportivo omette la descrizione del problema.

1. Il Sistema segnala che il campo descrizione è obbligatorio.

2. Lo Sportivo inserisce il testo o annulla l'operazione.

Modello di dominio

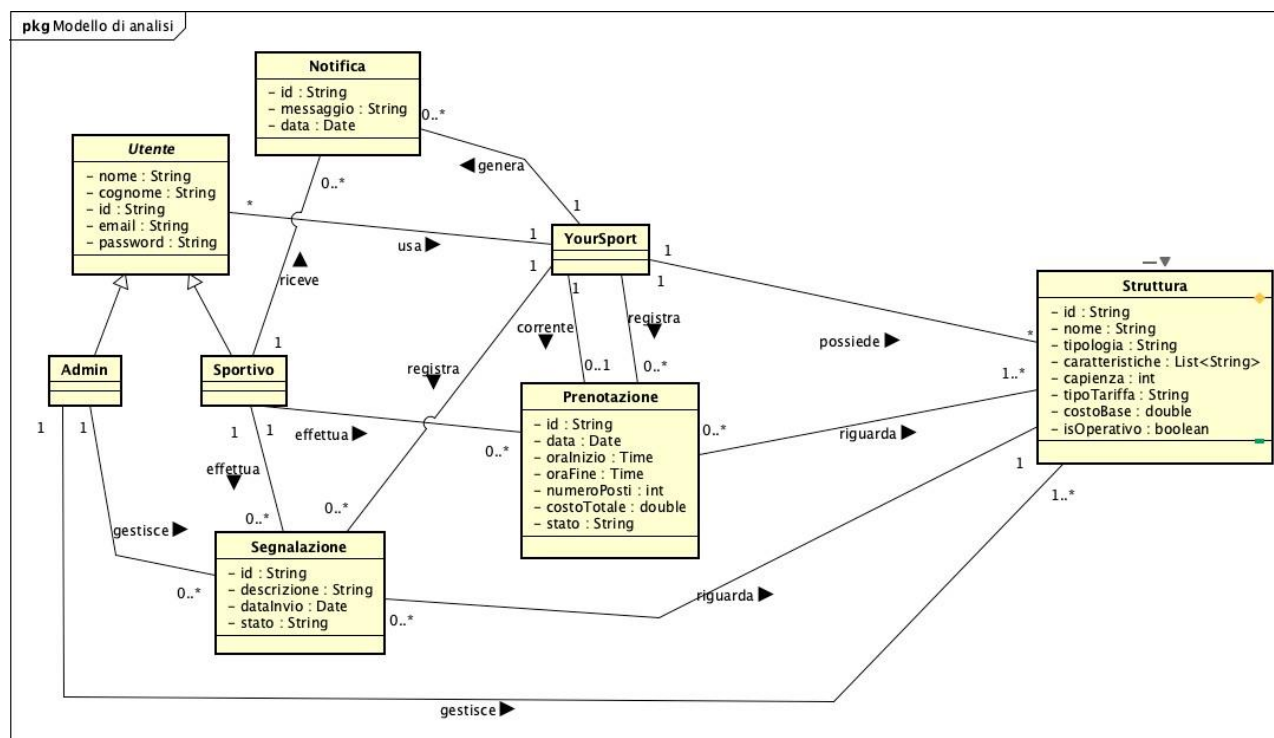
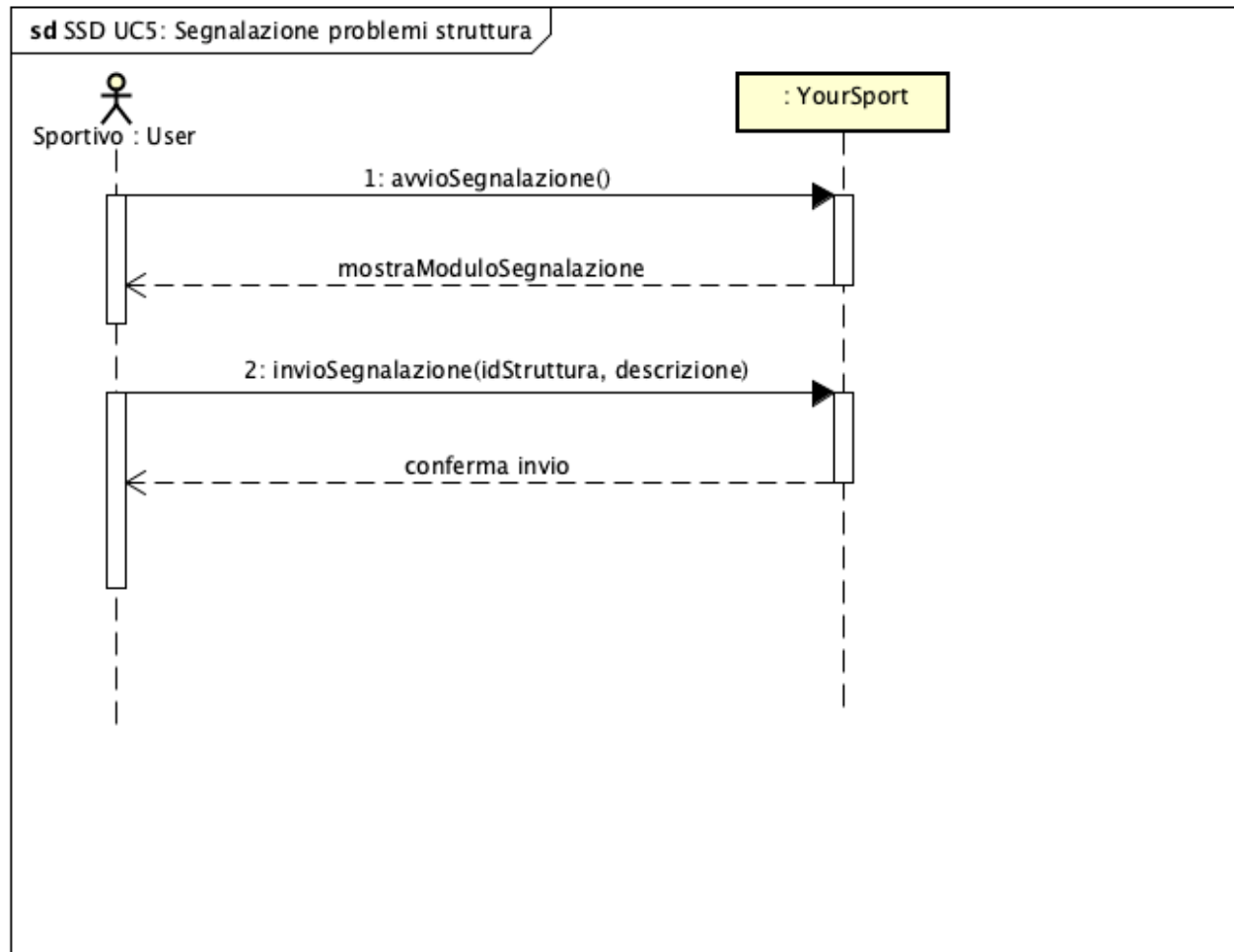


Diagramma di sequenza di sistema



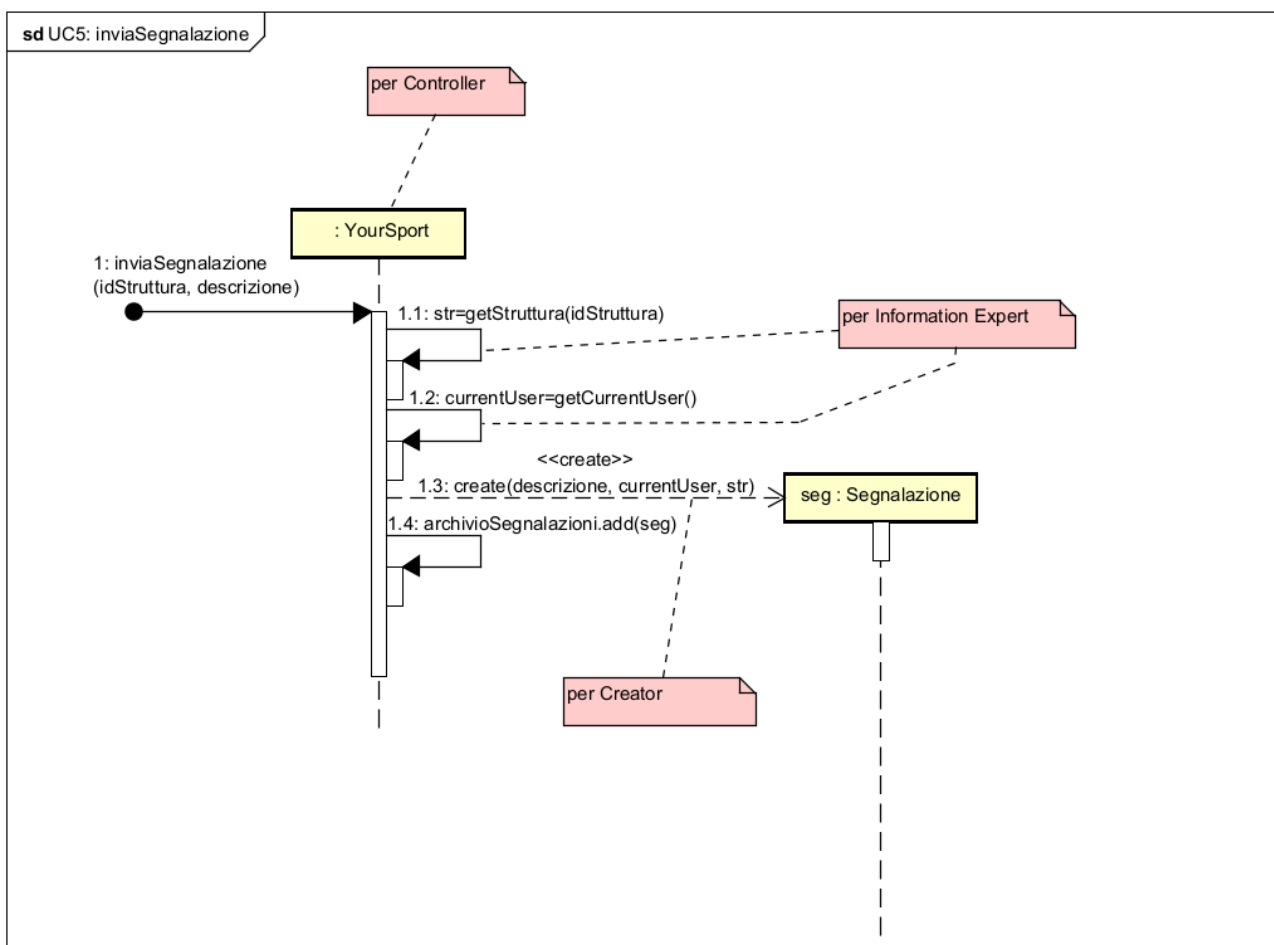
Contratti delle operazioni

-UC5

Operazioni	inviaSegnalazione(idStruttura: String, descrizione: String)
Riferimenti	UC5: Segnalazione problemi struttura
Pre-condizioni	-Lo Sportivo corrente è autenticato nel sistema. - Esiste un'istanza di Struttura il cui identificativo corrisponde a idStruttura

Post-condizioni	- È stata creata una nuova istanza <i>seg</i> della classe Segnalazione. - Gli attributi <i>descrizione</i>, <i>data</i> e <i>stato</i> dell'istanza <i>seg</i> sono stati inizializzati. - L'istanza <i>seg</i> è stata associata allo Sportivo corrente. - L'istanza <i>seg</i> è stata associata alla Struttura corrispondente.
-----------------	---

Diagrammi di sequenza



Testing

Test 1: Invia Segnalazione con Successo

Descrizione: Simuliamo l'invio di un ticket di segnalazione guasto (es. "Rete da tennis strappata") per una struttura realmente esistente, da parte di uno sportivo.

Scopo: Verificare che il sistema istanzi correttamente l'oggetto Segnalazione, assegnandogli un ID, la data odierna e impostando lo stato iniziale su "Aperta". Verifica inoltre le associazioni modellate nel DCD.

Realizzazione: Il test invoca il metodo `inviaSegnalazione` e, tramite asserzioni (`assertNotNull` e `assertEquals`), ispeziona l'oggetto restituito. Naviga le relazioni dell'oggetto per certificare che i puntatori leghino esattamente l'utente loggato e la struttura indicata.

Test 2: Errore Invia Segnalazione (Struttura Inesistente)

Descrizione: Simuliamo un errore in cui l'utente tenta di segnalare un guasto inserendo l'ID di un campo sportivo che non esiste nel database (es. "S_INVENTATO_99").

Scopo: Garantire l'integrità dei dati. Il sistema deve validare l'input e rifiutare la creazione del ticket se non può collegarlo a una struttura reale.

Realizzazione: Il test passa un ID falso al controller. Tramite `assertThrows`, si intercetta la `IllegalArgumentException` prevista. L'asserzione ispeziona il messaggio dell'eccezione per confermare che il rifiuto sia scattato perché la struttura non è stata trovata nell'archivio.

UC6: Gestione segnalazioni.

Nome caso d'uso	UC6: Gestione segnalazioni
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Admin
Parti interessate e interessi	Admin: vuole monitorare lo stato dei messaggi di segnalazione
Pre-condizioni	L'Admin è autenticato
Garanzie di successo	Lo stato della segnalazione viene aggiornato correttamente

Scenario principale di successo	1. L'Admin accede al pannello "Centro Segnalazioni". 2. Il Sistema mostra l'elenco di tutte le segnalazioni attive. 3. L'Admin seleziona una specifica segnalazione per visualizzarne i dettagli (Autore, Struttura, Descrizione). 4. L'Admin aggiorna lo stato della segnalazione (es. da "Aperta" a "In Lavorazione" o "Risolta"). 5. Il Sistema salva la modifica e aggiorna la visualizzazione. 6. L'Admin indica di aver terminato la gestione.
Estensioni	*a. In qualsiasi momento il sistema fallisce. 1. L'admin riavvia il software e ripristina lo stato. 2a. Non sono presenti segnalazioni nel sistema. 1. Il Sistema mostra un messaggio informativo ("Nessuna segnalazione presente").

pkg Modello di analisi

```
classDiagram
    class Utente {
        +nome : String
        +cognome : String
        +id : String
        +email : String
        +password : String
    }
    class Admin
    class Sportivo
    class Notifica {
        +id : String
        +messaggio : String
        +data : Date
    }
    class YourSport {
        +id : String
        +data : Date
        +oraInizio : Time
        +oraFine : Time
        +numeroPosti : int
        +costoTotale : double
        +stato : String
    }
    class Prenotazione {
        +id : String
        +data : Date
        +oraInizio : Time
        +oraFine : Time
        +numeroPosti : int
        +costoTotale : double
        +stato : String
    }
    class Segnalazione {
        +id : String
        +descrizione : String
        +dataInvio : Date
        +stato : String
    }
    class Struttura {
        +id : String
        +nome : String
        +tipologia : String
        +caratteristiche : List<String>
        +capienza : int
        +tipoTariffa : String
        +costoBase : double
        +isOperativo : boolean
    }

    Utente <|-- Admin
    Utente <|-- Sportivo
    Utente "0..*" -- "0..*" Notifica : genera
    Utente "0..*" -- "1" YourSport : usa
    Utente "1" -- "1" YourSport : registra
    Utente "1" -- "0..1" Prenotazione : corrente
    Utente "1" -- "0..*" Segnalazione : gestisce
    Utente "1" -- "1..*" Struttura : possiede
    YourSport "1" -- "1" Prenotazione : effettua
    YourSport "1" -- "0..*" Segnalazione : riguarda
    Prenotazione "0..*" -- "0..*" Segnalazione : gestisce
    Segnalazione "0..*" -- "1..*" Struttura : riguarda
```

```
sequenceDiagram
    actor Admin as Admin : User
    participant YourSport as : YourSport

    Admin->>YourSport: 1: mostraSegnalazioni()
    activate YourSport
    YourSport-->>Admin: elencoSegnalazioni
    deactivate YourSport

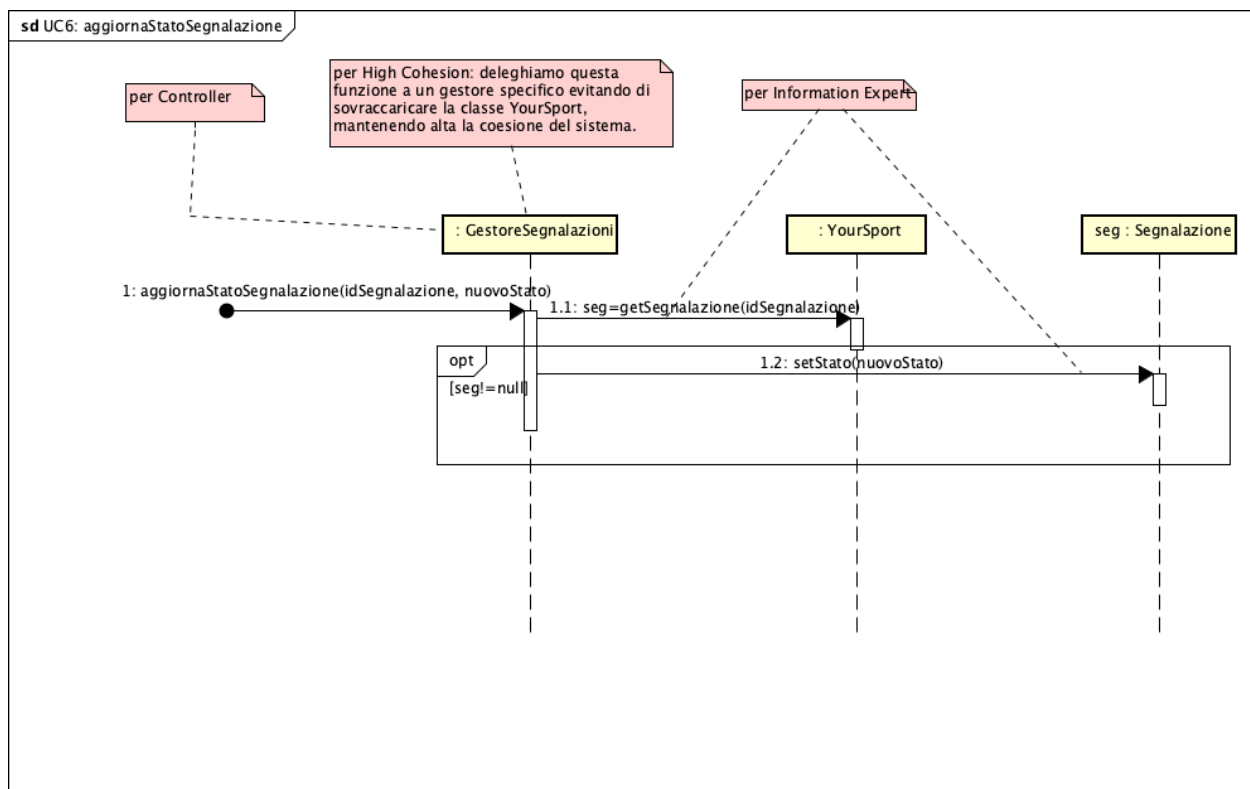
    loop [finché ci sono segnalazioni da aggiornare]
        Admin->>YourSport: 2: aggiornaStatoSegnalazione(idSegnalazione, nuovoStato)
        activate YourSport
        YourSport-->>Admin: conferma
        deactivate YourSport
    end
```

Contratti delle operazioni

-UC6

Operazioni	aggiornaStatoSegnalazione(idSegnalazione : String, nuovoStato: String)
Riferimenti	UC6: Gestione segnalazioni
Pre-condizioni	- L'utente corrente è autenticato nel sistema come Admin. - Esiste un'istanza di Segnalazione il cui identificativo corrisponde a idSegnalazione
Post-condizioni	- L'attributo <i>stato</i> dell'istanza Segnalazione corrispondente è stato modificato e impostato al valore del parametro nuovoStato.

Diagrammi di sequenza



Testing

Test 1: Lettura dell'Elenco Segnalazioni (Visualizzazione)

Descrizione: Simuliamo la richiesta dell'Amministratore di visualizzare a schermo l'intero storico dei guasti segnalati dagli sportivi.

Scopo: Dimostrare che il sistema riesca a recuperare correttamente l'archivio globale delle segnalazioni, assicurandosi che la connessione ai dati funzioni e che non vengano restituite liste nulle.

Realizzazione: Il test invoca il metodo `mostraSegnalazioni()`. Attraverso una serie di asserzioni (`assertNotNull`, `assertFalse`), verifica che la lista esista e non sia vuota. Successivamente, utilizza un ciclo `for` per ispezionare l'elenco e certificare (`assertTrue`) che la specifica segnalazione creata durante il setup iniziale del test sia fisicamente e correttamente presente al suo interno.

Test 2: Aggiornamento Stato Segnalazione (Successo)

Descrizione: Simuliamo l'intervento dell'Amministratore che prende in carico un ticket esistente, modificandone lo stato (ad esempio, passando da "Aperta" a "In risoluzione").

Scopo: Verificare il corretto funzionamento della logica di aggiornamento (il "cammino felice"). Il sistema deve individuare la segnalazione corretta e autorizzarla a sovrascrivere il proprio stato interno.

Realizzazione: Dopo un controllo di sicurezza per garantire che lo stato di partenza sia "Aperta", il test chiama il metodo `aggiornaStatoSegnalazione(...)` con parametri validi. La validazione finale si divide in due parti: prima si verifica che il metodo confermi il successo dell'operazione restituendo `true`; poi, ispezionando direttamente l'oggetto in memoria (`assertEquals`), si certifica che il suo attributo di stato sia stato effettivamente rimpiazzato con il nuovo valore.

Test 3: Aggiornamento Fallito per ID Inesistente (Riquadro [opt])

Descrizione: Simuliamo un errore di input da parte dell'Admin, che tenta di aggiornare lo stato di una segnalazione digitando un ID non presente al sistema (es. "ID_FALSO_INVENTATO_999").

Scopo: Testare la robustezza del sistema e l'esatta esecuzione del blocco opzionale (opt) modellato nell'UML. Se l'ID non viene trovato, il flusso deve interrompersi senza generare crash e senza corrompere i dati di altre segnalazioni.

Realizzazione: Il test passa deliberatamente un ID falso al controller. Tramite un'asserzione `assertFalse`, verifica che il metodo intercetti il problema e respinga la modifica. Infine, esegue un controllo di sicurezza cruciale sull'oggetto originale: assicura che il suo stato sia rimasto intatto ("Aperta"), dimostrando che il sistema ha protetto i dati da sovrascritture accidentali.

UC7: Sospensione e Riattivazione Struttura.

Nome caso d'uso	UC7: Sospensione e Riattivazione Struttura
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Admin
Parti interessate e interessi	Admin: deve chiudere una struttura e gestire i conflitti con le prenotazioni esistenti.
Pre-condizioni	L'Admin ha rilevato un guasto critico (tramite UC6)
Garanzie di successo	La struttura viene resa non operativa, le prenotazioni future impattate vengono annullate e i relativi sportivi vengono avvisati.
Scenario principale di successo	1. L'Admin seleziona la struttura da dichiarare fuori servizio. 2. Il Sistema richiede conferma per la sospensione dell'operatività. 3. L'Admin conferma la sospensione. 4. Il Sistema imposta l'attributo "isOperativo" della struttura a <i>False</i>. 5. Il Sistema identifica automaticamente tutte le prenotazioni future associate a quella struttura. 6. Il Sistema annulla forzatamente le prenotazioni trovate. 7. Il Sistema genera una notifica automatica (UC8) per ogni sportivo coinvolto spiegando il motivo del disservizio. 8. Il Sistema conferma all'Admin il numero di prenotazioni annullate e i messaggi inviati.

Estensioni

***a.** In qualsiasi momento il sistema fallisce.

1. L'admin riavvia il software e ripristina lo stato.

3a. L'Admin seleziona una struttura che è attualmente nello stato "Fuori Servizio".

1. Il Sistema chiede conferma per la riattivazione.

2. L'Admin conferma.

3. Il Sistema imposta lo stato della struttura come "Operativa", rendendola nuovamente disponibile per le prenotazioni (UC2).

5a. Non ci sono prenotazioni future per quella struttura.

1. Il Sistema disattiva semplicemente l'operatività della struttura e termina lo scenario con successo.

Modello di dominio

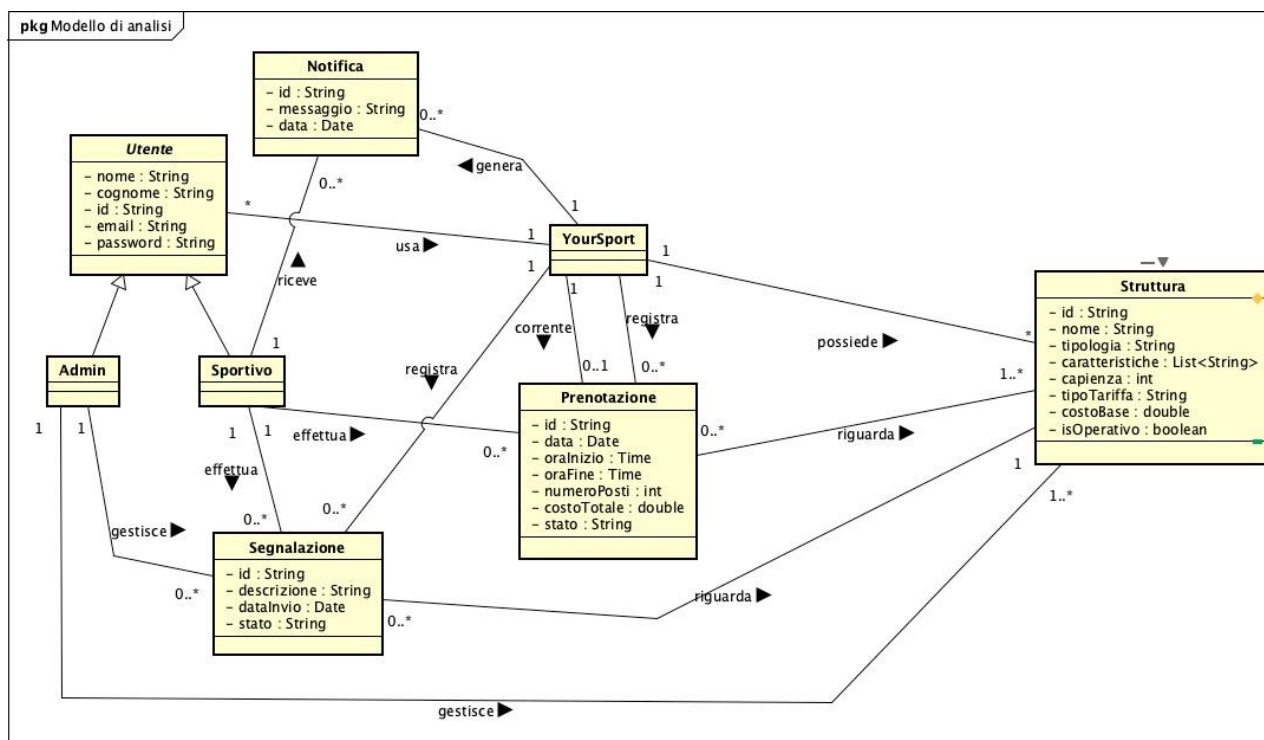
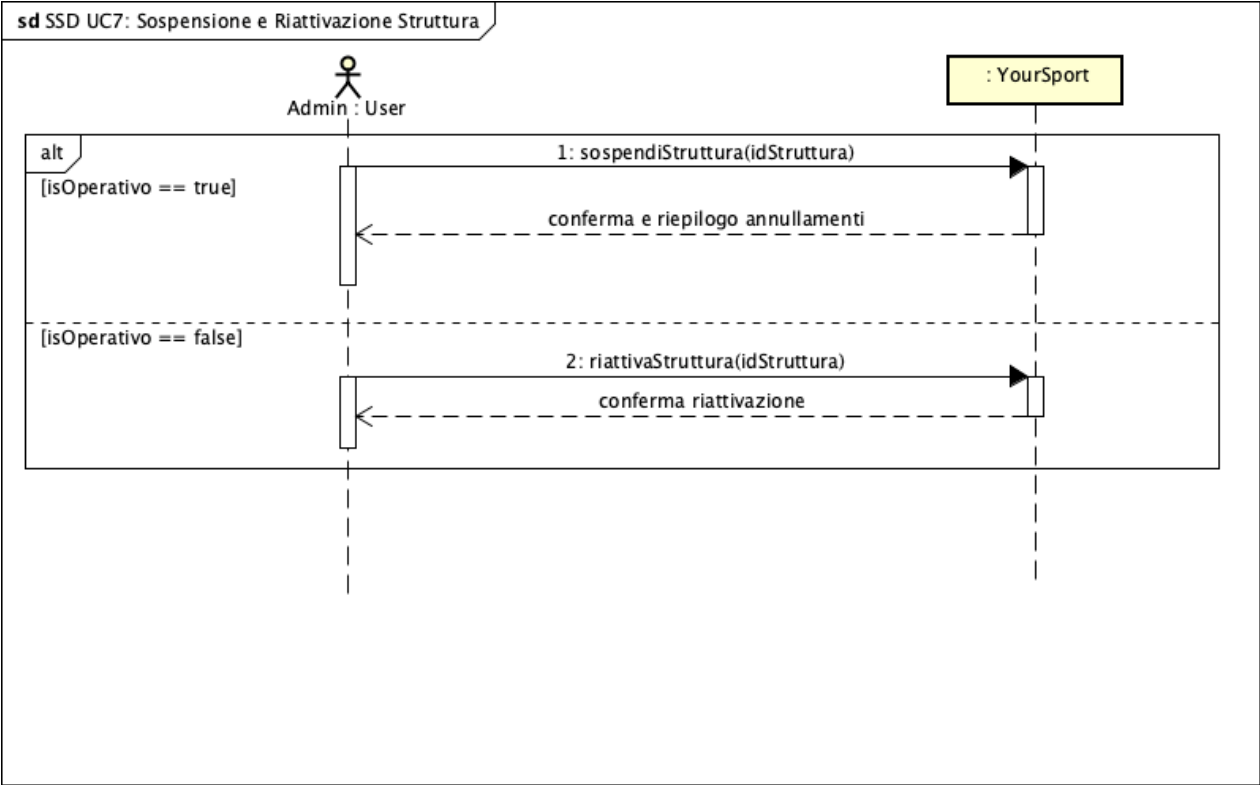


Diagramma di sequenza di sistema



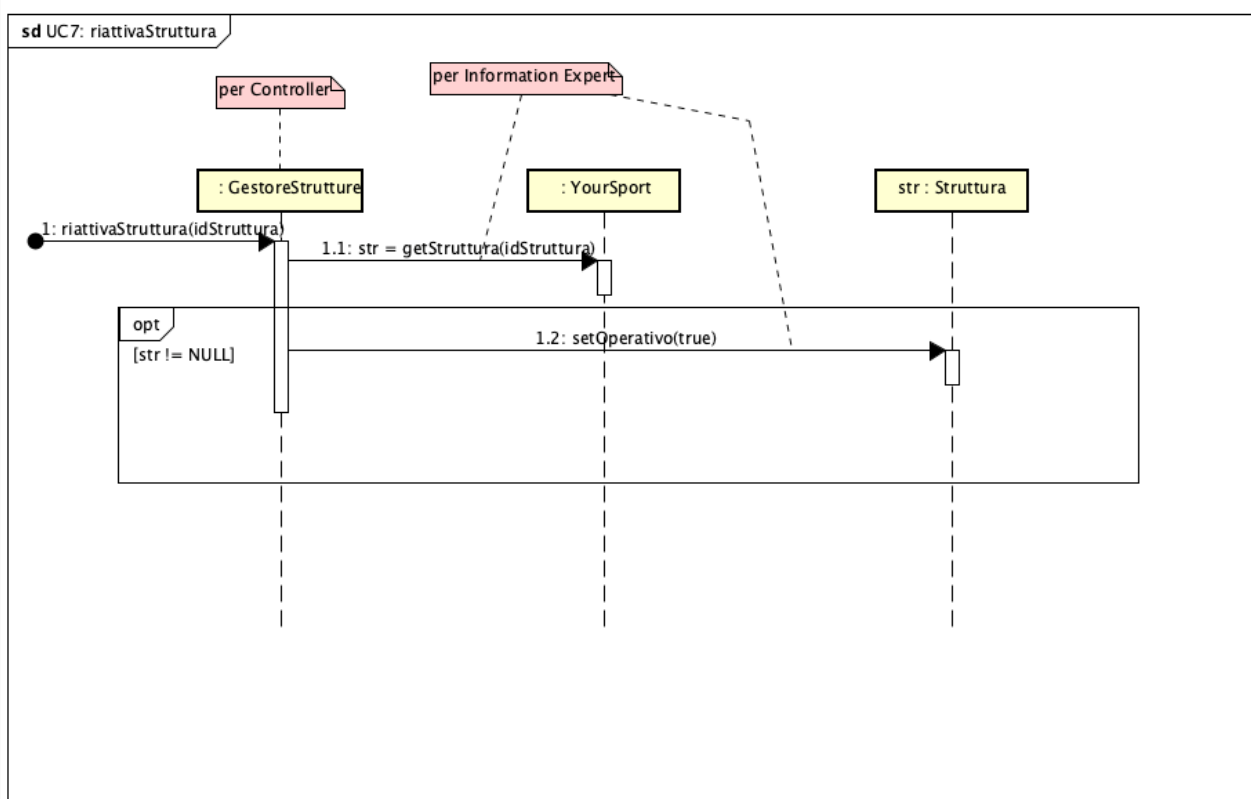
Contratti delle operazioni

Operazioni	sospendiStruttura(idStruttura: String)
Riferimenti	UC7: Sospensione e Riattivazione Struttura.
Pre-condizioni	- L'utente corrente è autenticato nel sistema come Admin. - Esiste un'istanza di Struttura il cui identificativo corrisponde a idStruttura.
Post-condizioni	- L'attributo <i>isOperativo</i> dell'istanza Struttura corrispondente è stato impostato a <i>false</i>. - L'attributo <i>stato</i> di tutte le istanze di Prenotazione future associate a tale Struttura è stato modificato in "Annullata". - È stata creata una nuova istanza di Notifica per ogni Sportivo la cui prenotazione è stata annullata forzatamente.

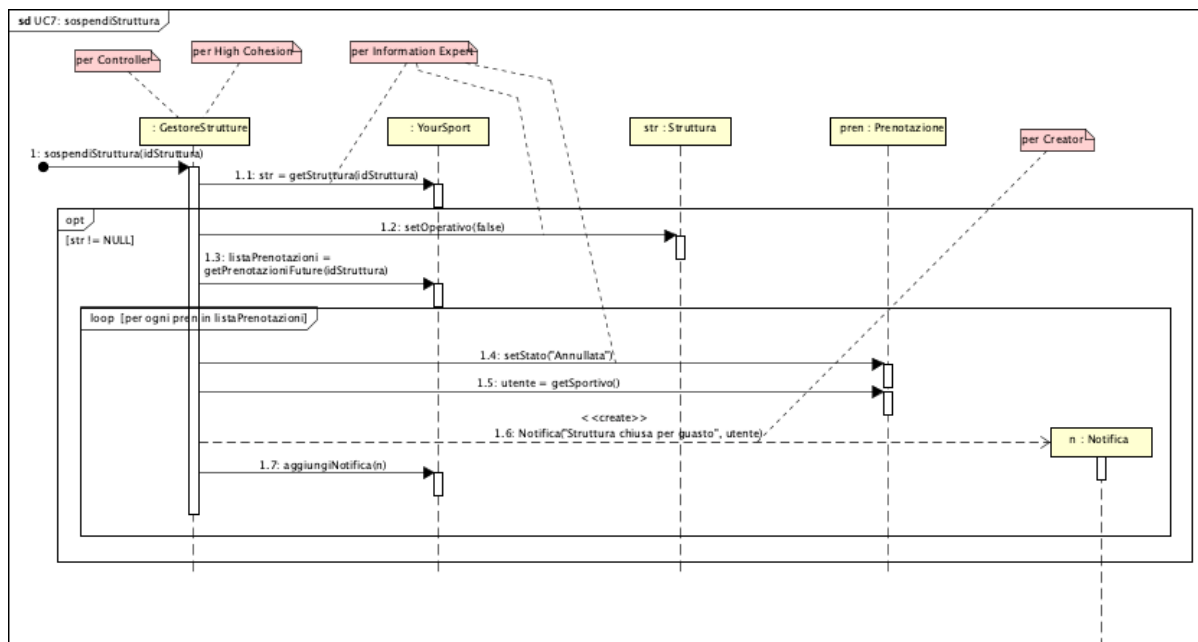
Operazioni	riattivaStruttura(idStruttura: String)
Riferimenti	UC7: Sospensione e Riattivazione Struttura.
Pre-condizioni	Esiste un'istanza di Struttura s associata all'identificatore idStruttura, e il suo attributo isOperativo è attualmente false.
Post-condizioni	L'attributo isOperativo dell'istanza s è stato modificato a true.

Diagrammi di sequenza

-riattivaStruttura



-sospendiStruttura



Testing

Test 1: Sospensione Struttura con Successo

Descrizione: Simuliamo l'azione dell'Amministratore che decide di sospendere (mettere fuori servizio) una struttura sportiva esistente (es. la struttura "S1").

Scopo: Verificare l'esatta esecuzione dell'intero flusso: la disattivazione fisica della struttura, l'annullamento di tutte le prenotazioni future associate e la generazione automatica delle notifiche per avvisare gli sportivi coinvolti dal disservizio.

Realizzazione: Il test invoca il metodo sospendiStruttura. Oltre a confermare l'esito positivo (assertTrue), esegue tre controlli di stato incrociati: 1) accerta che l'attributo di operatività della struttura sia diventato false; 2) verifica che lo stato di una prenotazione futura di test sia passato ad "Annullata"; 3) ispeziona l'archivio notifiche globale per assicurarsi che il messaggio di avviso sia stato creato e associato al corretto destinatario (messaggi 1.6 e 1.7 dell'SD).

Test 2: Sospensione Fallita per ID Inesistente (Blocco opt)

Descrizione: Simuliamo un errore di input in cui l'Admin tenta di sospendere una struttura fornendo un id non presente a sistema (es. "ID_FALSO_999").

Scopo: Testare la solidità del sistema. Il controller deve intercettare l'anomalia (bloccando l'esecuzione del riquadro opt dell'UML).

Realizzazione: Passando l'ID inesistente, il test verifica (assertFalse) che il metodo respinga l'operazione, dopo effettua un controllo di sicurezza per garantire che le strutture valide (come la "S1") siano rimaste regolarmente operative.

Test 3: Riattivazione Struttura con Successo (Scenario Alternativo)

Descrizione: Simuliamo l'operazione di ripristino, in cui l'Amministratore riaccende una struttura precedentemente posta fuori servizio, rendendola nuovamente disponibile.

Scopo: Dimostrare che il sistema gestisca correttamente il cambio di stato inverso, rendendo la risorsa di nuovo interfacciabile con i flussi di prenotazione degli sportivi.

Realizzazione: Il test prepara l'ambiente forzando manualmente lo spegnimento di una struttura di test ("S2"). Successivamente, invoca `riattivaStruttura` e certifica tramite asserzioni che il metodo completi l'operazione con successo e che l'attributo interno operativo della struttura sia tornato a `true`.

Test 4: Riattivazione Fallita per ID Inesistente

Descrizione: Simuliamo il tentativo di riattivare una struttura fornendo un ID finto ("STRUTTURA_GHOST").

Scopo: Assicurare che anche il flusso di riattivazione includa la validazione dei dati e sia protetto da input errati o malformati.

Realizzazione: Il test passa il parametro errato al gestore e verifica (`assertFalse`) che il sistema valuti negativamente la condizione iniziale e respinga la richiesta.

Iterazione 5

Durante l'Iterazione 5, lo sviluppo ha completato il quadro applicativo concentrandosi sulle aree personali e sull'amministrazione avanzata. I lavori hanno riguardato l'implementazione del **Caso d'Uso 8 (Visualizza le notifiche)**, del **Caso d'Uso 9 (Visualizza statistiche incassi previsti)** e del **Caso d'Uso 10 (Gestione Profilo Personale)**.

I pattern utilizzati sono:

- **Controller / Pure Fabrication (GRASP):** Abbiamo introdotto un GestoreNotifiche dedicato per dirigere il flusso dell'UC8, evitando così di centralizzare tutto sulla classe YourSport. Per le operazioni globali di UC9 e UC10, YourSport ha continuato ad agire come Controller primario.
- **Information Expert (GRASP):** Le responsabilità sono state distribuite interrogando l'esperto corretto per ogni operazione: YourSport fornisce le istanze filtrate globali, l'oggetto Prenotazione è designato per restituire il proprio costo totale alla funzione statistica (UC9) e l'oggetto Sportivo possiede i metodi *setter* per manipolare il proprio stato interno (UC10).
- **High Cohesion (GRASP):** Questo pattern è stato esplicitamente applicato nella progettazione dell'UC9 delegando le funzioni minori all'oggetto Prenotazione, al fine di non sovraccaricare le classi di coordinamento e mantenere il sistema coeso e facilmente manutenibile.

UC8: Visualizza le notifiche.

Nome caso d'uso	UC8: Visualizza le notifiche
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Sportivo
Parti interessate e interessi	Sportivo: Vuole leggere gli avvisi riguardo i disservizi.
Pre-condizioni	Lo sportivo è autenticato
Garanzie di successo	Lo Sportivo visualizza correttamente l'elenco delle comunicazioni e dei disservizi a lui destinati
Scenario principale di successo	1. Lo Sportivo richiede al Sistema di accedere all'area delle notifiche. 2. Il Sistema recupera l'archivio delle notifiche associate a quello Sportivo. 3. Il Sistema mostra l'elenco delle notifiche, riportando per ciascuna la data e il messaggio. 4. Lo Sportivo prende visione delle notifiche. 5. Lo Sportivo esce dalla sezione.

Estensioni	a. In qualsiasi momento il sistema fallisce. 1. Lo sportivo riavvia il software e ripristina lo stato. 2a. Non ci sono notifiche presenti per quello Sportivo. 1. Il Sistema mostra il messaggio "Nessuna notifica presente" o "La tua bacheca è vuota". Lo scenario termina con successo.
-------------------	--

Modello di dominio

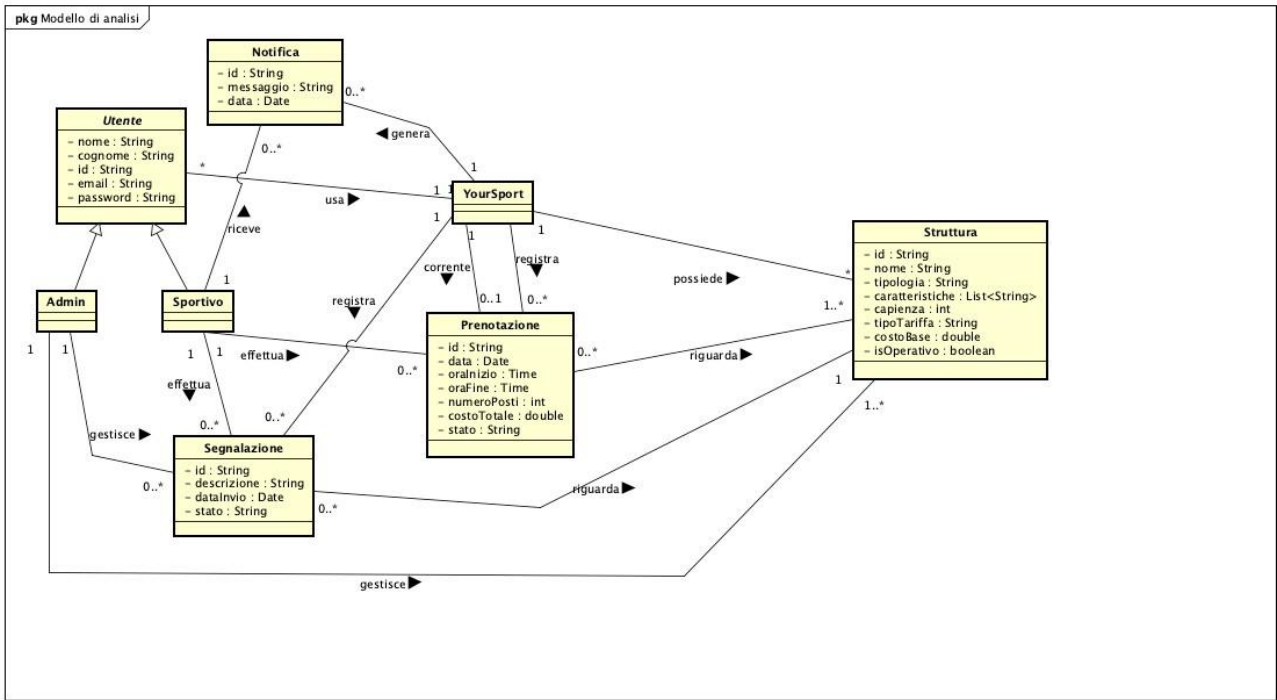
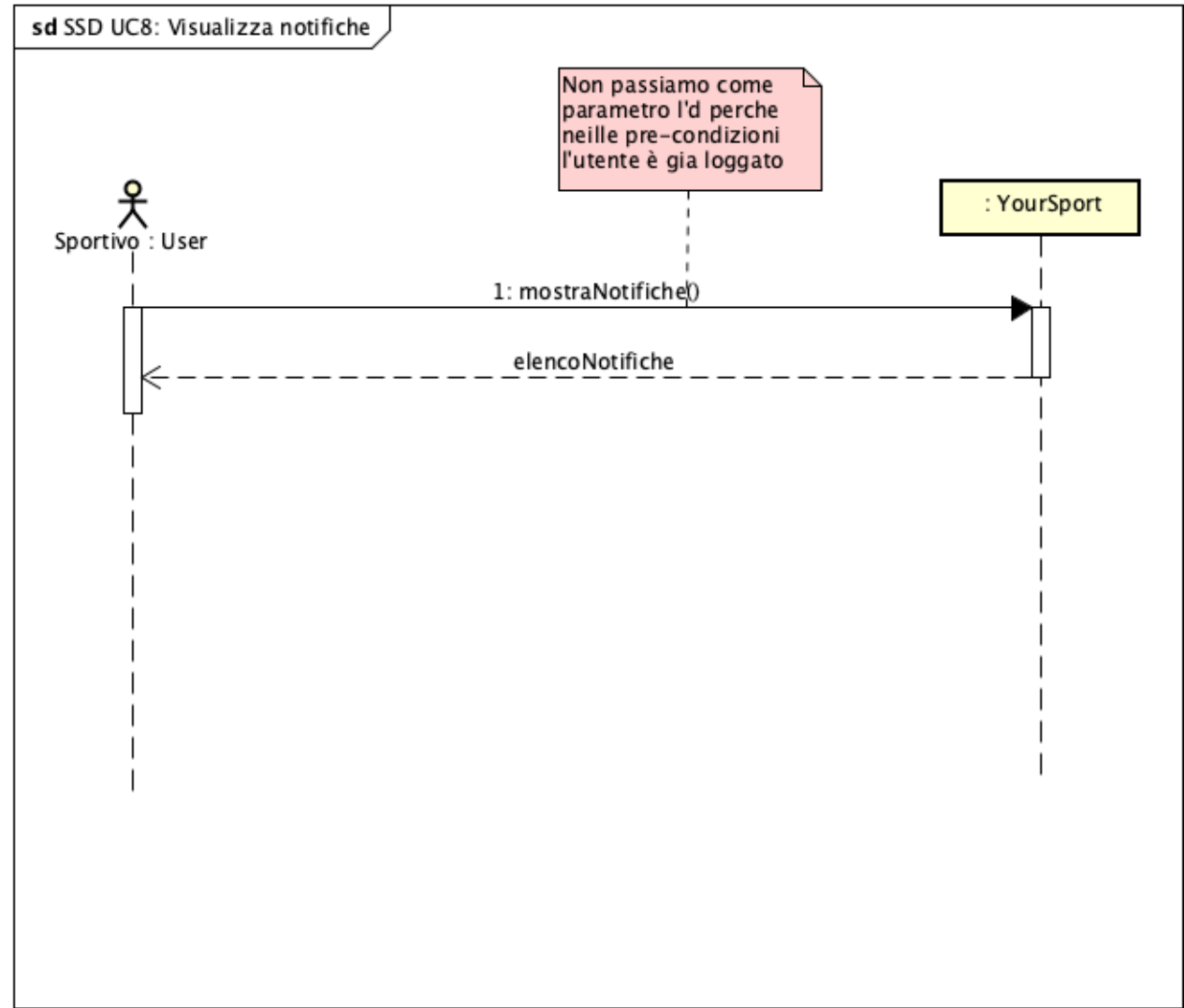
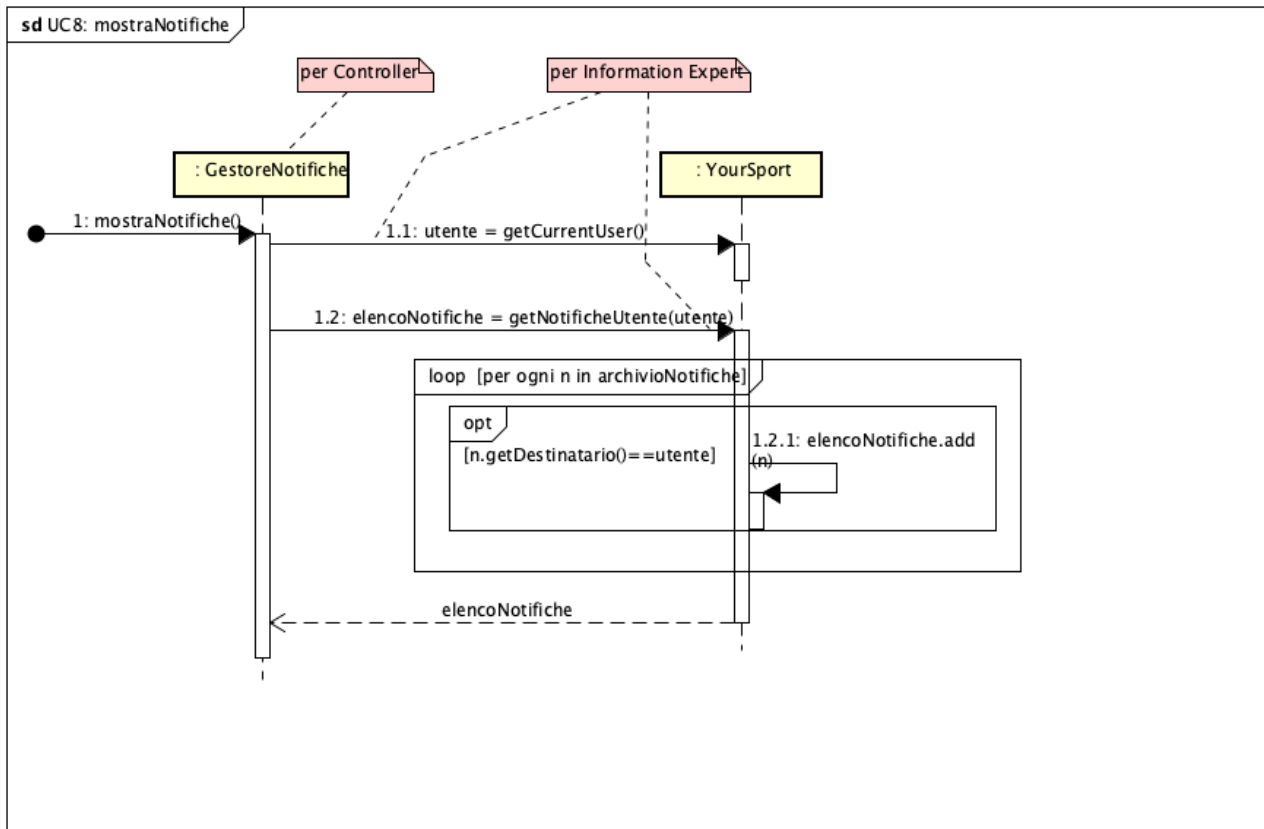


Diagramma di sequenza di sistema



Diagrammi di sequenza



Testing

Test 1: Visualizzazione Notifiche Personali (Successo e Filtro)

Descrizione: Simuliamo la presenza di due utenti distinti nel sistema (es. Mario e Luigi) con notifiche destinate a entrambi, e verifichiamo cosa vede Mario quando accede alla sua bacheca.

Scopo: Assicurarsi che il sistema recuperi correttamente i messaggi e, soprattutto, applichi il filtro di privacy. Il sistema deve mostrare allo sportivo solo le notifiche indirizzate a lui, nascondendo rigorosamente quelle degli altri utenti.

Realizzazione: Il test inietta nel sistema globale tre notifiche fittizie (due per Mario, una per Luigi). Dopo aver effettuato il login come Mario, invoca il controller per mostrare le notifiche e verifica tramite asserzioni che la lista restituita contenga esattamente due elementi e che il testo coincida con i messaggi originariamente destinati a Mario.

Test 2: Visualizzazione Bacheca Vuota (Assenza di Notifiche)

Descrizione: Simuliamo l'accesso alla sezione notifiche da parte di un utente che non ha mai subito annullamenti per disservizi o ricevuto altre comunicazioni dal sistema.

Scopo: Verificare la robustezza del sistema in assenza di dati. L'interrogazione dell'archivio non deve causare errori anomali (come `NullPointerException`), ma deve gestire in modo elegante l'assenza di notifiche per quello specifico utente.

Realizzazione: Il test registra un nuovo utente e ne effettua il login senza generare alcuna notifica a suo nome. Richiama il metodo di visualizzazione e verifica che il sistema restituisca un oggetto lista valido (non nullo) ma strettamente vuoto (dimensione pari a 0).

Test 3: Sicurezza e Accesso Non Autorizzato

Descrizione: Simuliamo un tentativo di richiesta delle notifiche quando non c'è nessun utente regolarmente connesso al sistema (es. sessione scaduta o tentativo di accesso esterno).

Scopo: Garantire che il Controller (GestoreNotifiche) protegga l'integrità del sistema. Se non rileva un utente autenticato, deve bloccare immediatamente la richiesta alla radice, evitando interrogazioni inutili al database e proteggendo i dati.

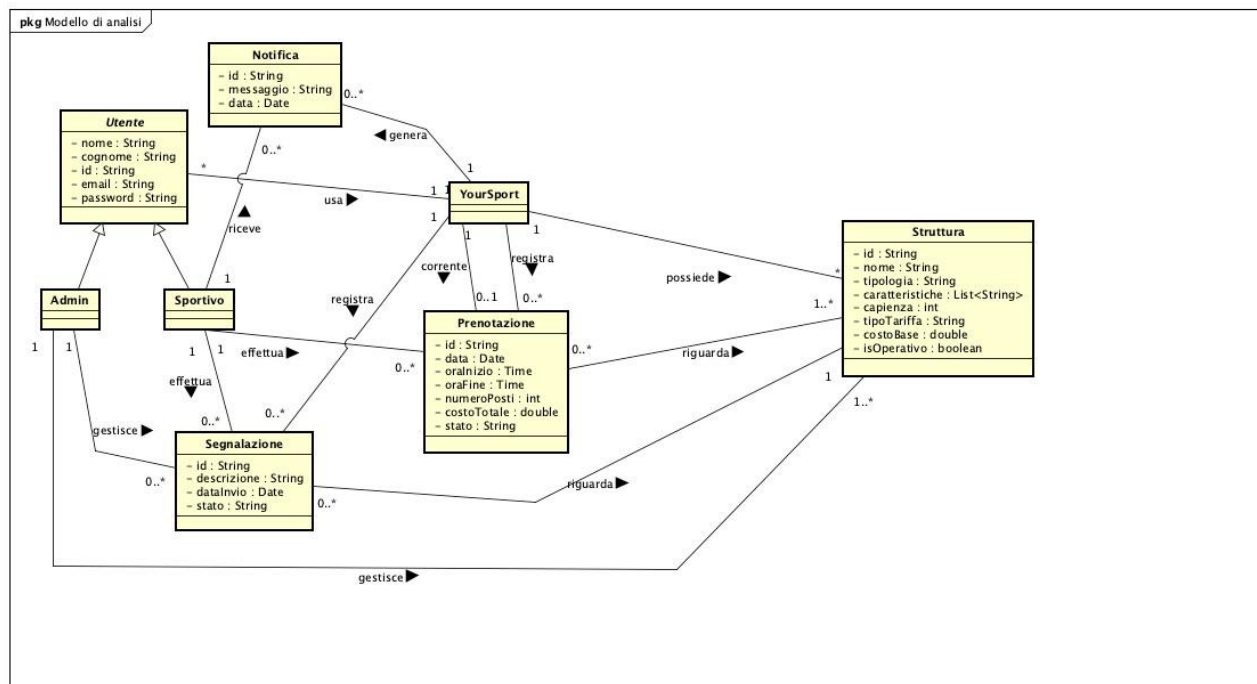
Realizzazione: Il test forza esplicitamente il logout, svuotando la sessione corrente (currentUser = null). Successivamente, invoca la richiesta di visualizzazione notifiche e verifica che il sistema respinga l'operazione restituendo un valore null (blocco di sicurezza).

UC9: Visualizza statistiche incassi previsti:

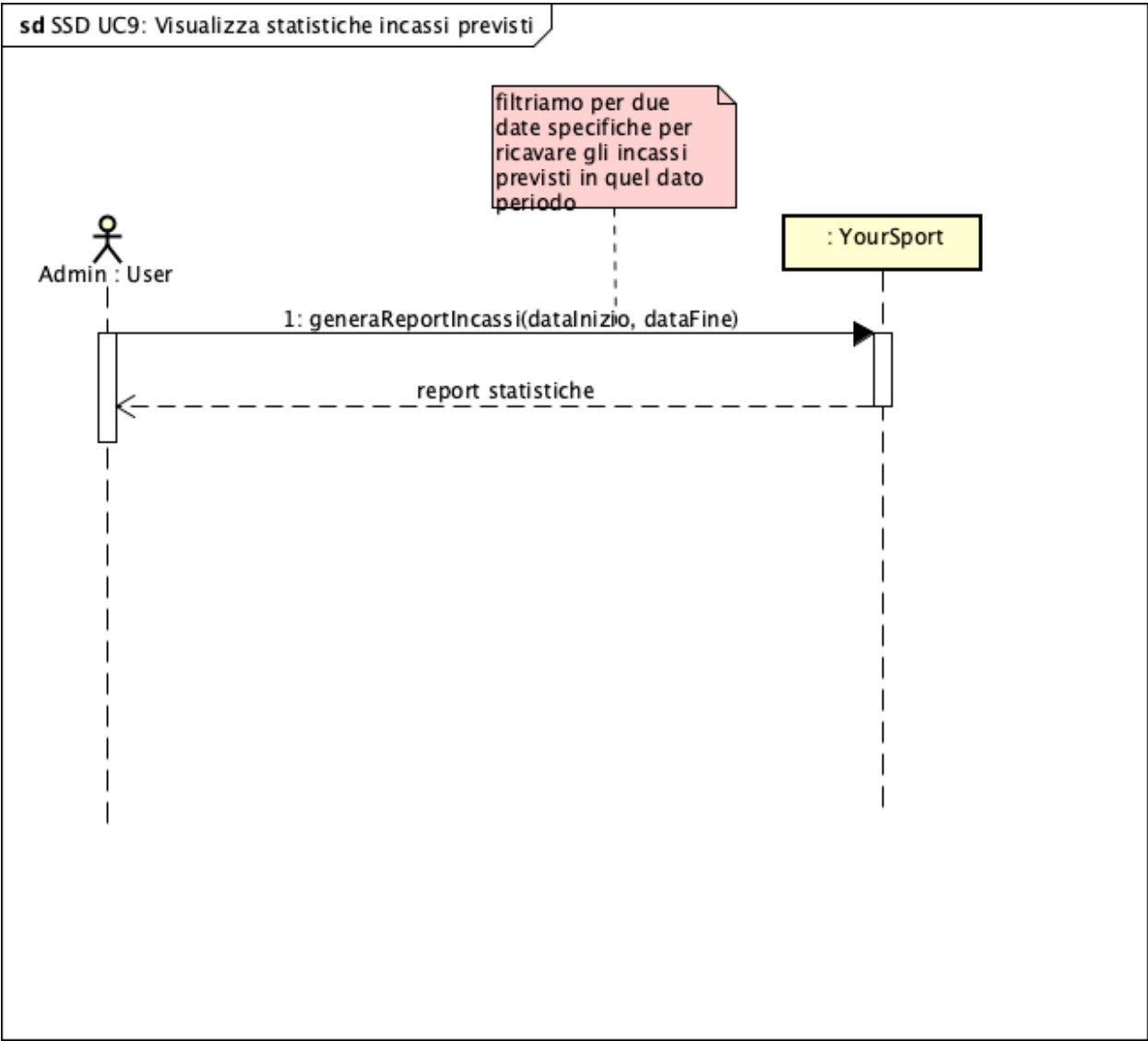
Nome caso d'uso	UC9: Visualizza statistiche incassi previsti
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Admin
Parti interessate e interessi	Admin: Deve consultare i dati aggregati relativi agli introiti delle prenotazioni per monitorare l'andamento economico e valutare la rendita delle diverse strutture dell'impianto
Pre-condizioni	L'admin è autenticato
Garanzie di successo	L'Admin visualizza un report degli incassi previsti calcolati in base alle prenotazioni registrate.

Scenario principale di successo	1. L'Admin seleziona la funzione per la visualizzazione delle statistiche degli incassi. 2. Il Sistema richiede di specificare il periodo di riferimento (es. mese o range di date). 3. L'Admin inserisce o seleziona il periodo desiderato. 4. Il Sistema elabora tutte le prenotazioni associate a quel periodo. 5. Il Sistema aggrega i dati calcolando l'incasso totale previsto e la rendita divisa per ogni singola Struttura. 6. Il Sistema mostra il report statistico elaborato.
Estensioni	a. In qualsiasi momento il sistema fallisce. 1. Lo sportivo riavvia il software e ripristina lo stato. 4a. Non sono presenti prenotazioni nel periodo selezionato. 1. Il Sistema mostra a video che non ci sono dati sufficienti per il periodo e riporta gli introiti a zero. Lo scenario termina con successo.

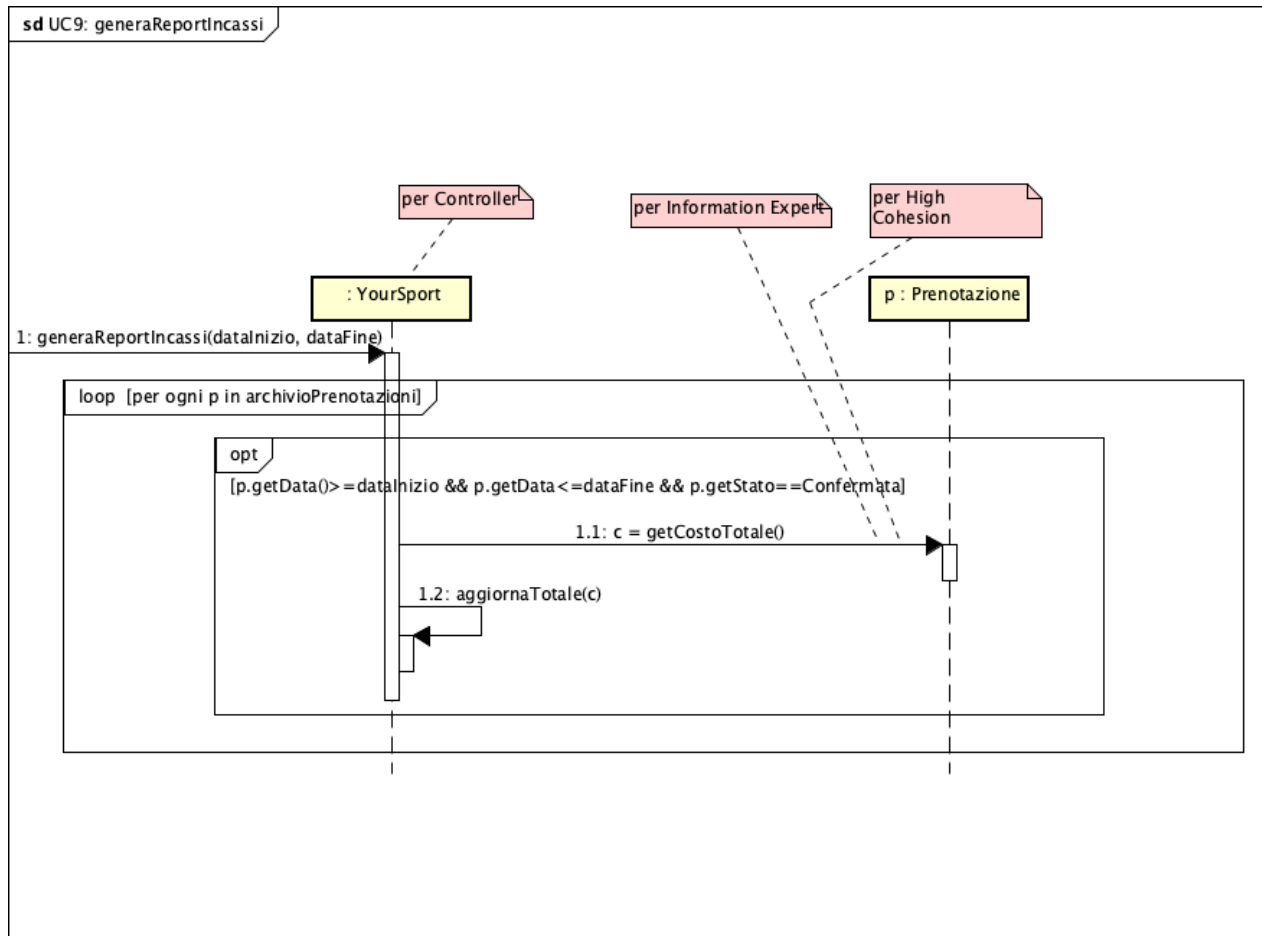
Modello di dominio



Diagrammi di sequenza di sistema



Diagrammi di sequenza



Testing

Test 1: Calcolo Incassi Previsti (Periodo Valido)

- **Descrizione:** Simuliamo la generazione di un report finanziario per un periodo di tempo specifico, inserendo preventivamente nel sistema una prenotazione valida e con stato "Confermata" che ricade esattamente in quell'arco temporale.
- **Scopo:** Assicurarsi che l'algoritmo scandagli correttamente l'archivio delle prenotazioni, filtri le date tramite le funzioni della classe `LocalDate` e calcoli il totale sommando solo le prenotazioni pertinenti al periodo d'interesse.
- **Realizzazione:** Il test crea oggetti reali (una `Struttura` e uno `Sportivo`) utilizzando i costruttori esatti del sistema e registra una prenotazione "Confermata" in una data definita. Successivamente, invoca il metodo `generaReportIncassi` passando come argomenti un intervallo di tempo che include quella data. Tramite un'asserzione di base (`assertTrue`), il test verifica matematicamente che il totale calcolato dal sistema sia maggiore di zero, a dimostrazione che la prenotazione è stata trovata e processata con successo.

Test 2: Calcolo Incassi (Periodo senza incassi o archivio vuoto)

- **Descrizione:** Simuliamo la richiesta di statistiche da parte dell'amministratore per un intervallo temporale passato in cui non è stata registrata alcuna prenotazione (oppure dove sono presenti solo prenotazioni "Annullate").
- **Scopo:** Verificare la robustezza del sistema in assenza di dati utili. Il ciclo di calcolo non deve generare errori bloccanti (come NullPointerException) e deve restituire un bilancio neutro.
- **Realizzazione:** Il test interroga il sistema passando un range di date relative a mesi precedenti, per i quali il database simulato è vuoto. Il test si conclude verificando in modo diretto e intuitivo (`assertTrue(totale == 0.0)`) che il sistema restituisca esattamente un importo pari a zero.

Test 3: Robustezza su Inserimento Date Invertite

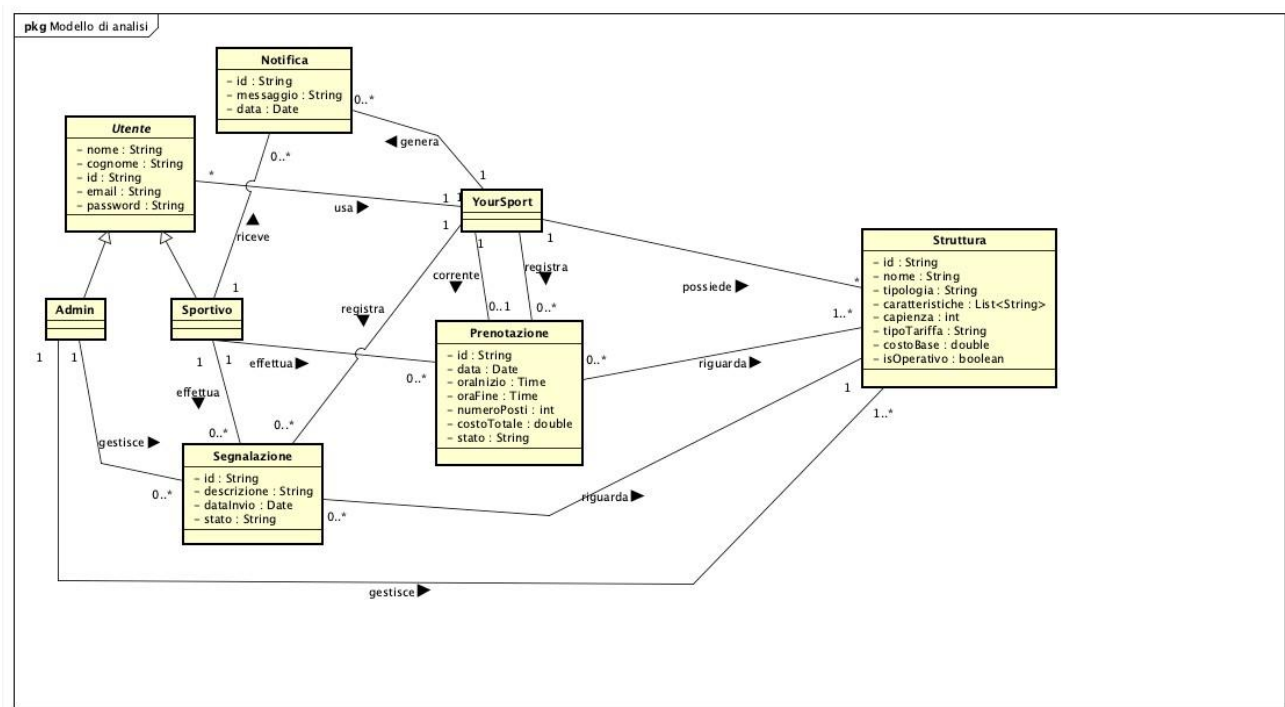
- **Descrizione:** Simuliamo un classico errore umano di input: l'amministratore tenta di generare il report inserendo una data di "fine periodo" cronologicamente antecedente alla data di "inizio periodo".
- **Scopo:** Garantire che l'algoritmo di Business Logic sia sicuro contro dati di input illogici. Il sistema deve gestire la situazione in modo silente, senza andare in crash.
- **Realizzazione:** Il test invoca il calcolo degli incassi passando, per esempio, come inizio "oggi" e come fine "ieri". Poiché la condizione logica all'interno del sistema richiede che una prenotazione sia strettamente compresa tra i due limiti, l'algoritmo ignorerà tutti i record, e il test verificherà con successo che il risultato restituito sia 0.0.

UC10: Gestione Profilo Personale:

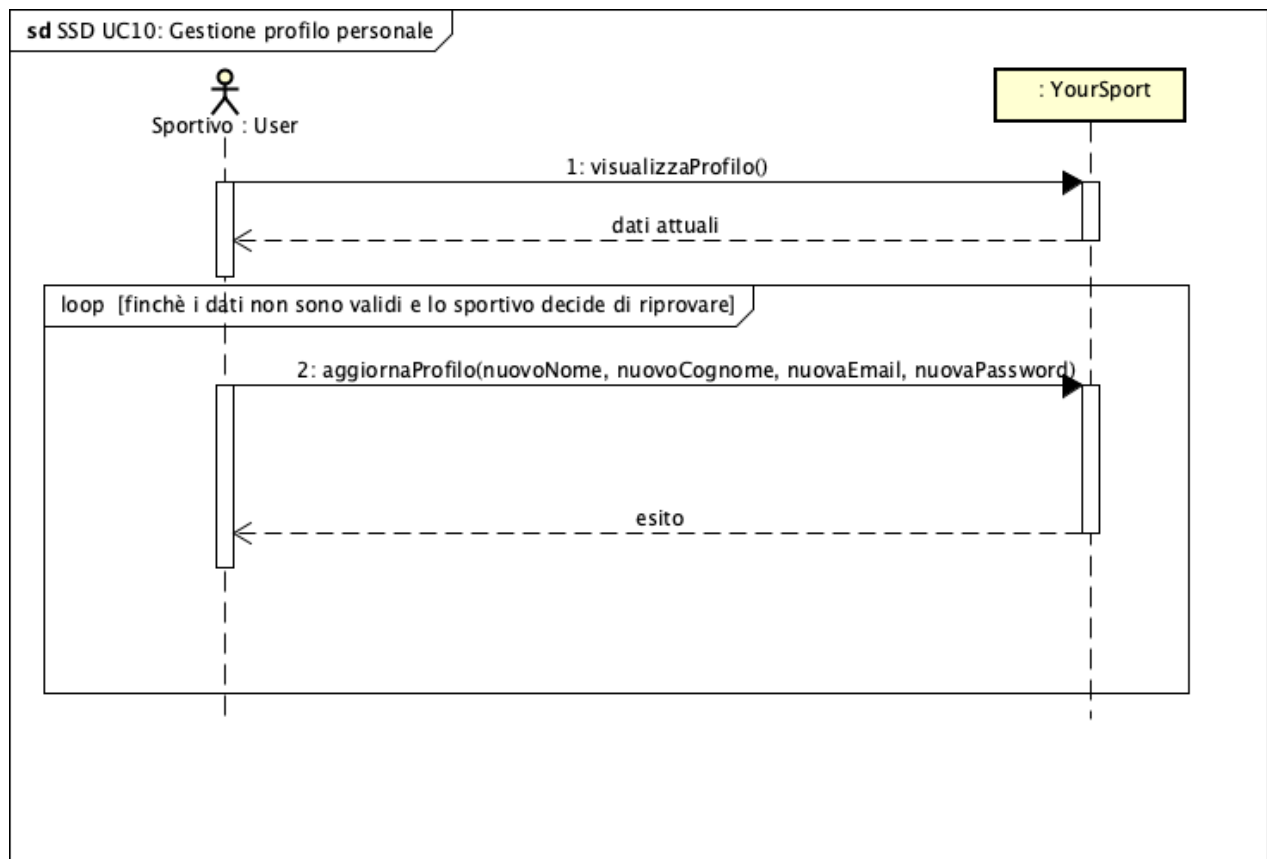
Nome caso d'uso	UC10: Gestione Profilo Personale
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Sportivo
Parti interessate e interessi	Sportivo: Vuole mantenere aggiornati i propri dati anagrafici e la propria password
Pre-condizioni	Lo sportivo è autenticato
Garanzie di successo	I dati anagrafici o le credenziali di accesso dello Sportivo vengono aggiornati e salvati correttamente

Scenario <u>principale</u> di successo	1. Lo Sportivo seleziona l'opzione per accedere alla propria area riservata (Profilo). 2. Il Sistema mostra i dati anagrafici e di contatto attuali dello Sportivo. 3. Lo Sportivo inserisce i nuovi dati o digita una nuova password. 4. Lo Sportivo conferma la volontà di salvare le modifiche. 5. Il Sistema valida i dati inseriti (es. verifica che i campi obbligatori non siano vuoti). 6. Il Sistema aggiorna il profilo dello Sportivo nel database. 7. Il Sistema mostra un messaggio di avvenuto aggiornamento con successo.
Estensioni	a. In qualsiasi momento il sistema fallisce. 1. Lo sportivo riavvia il software e ripristina lo stato. 5a. I dati inseriti dallo Sportivo non superano i controlli di validazione (es. email già in uso). 1. Il Sistema segnala specificamente quale dato è errato. 2. Il Sistema richiede allo Sportivo di correggere il dato. Lo scenario riprende dal passo 3.

Modello di dominio



Diagrammi di sequenza di sistema



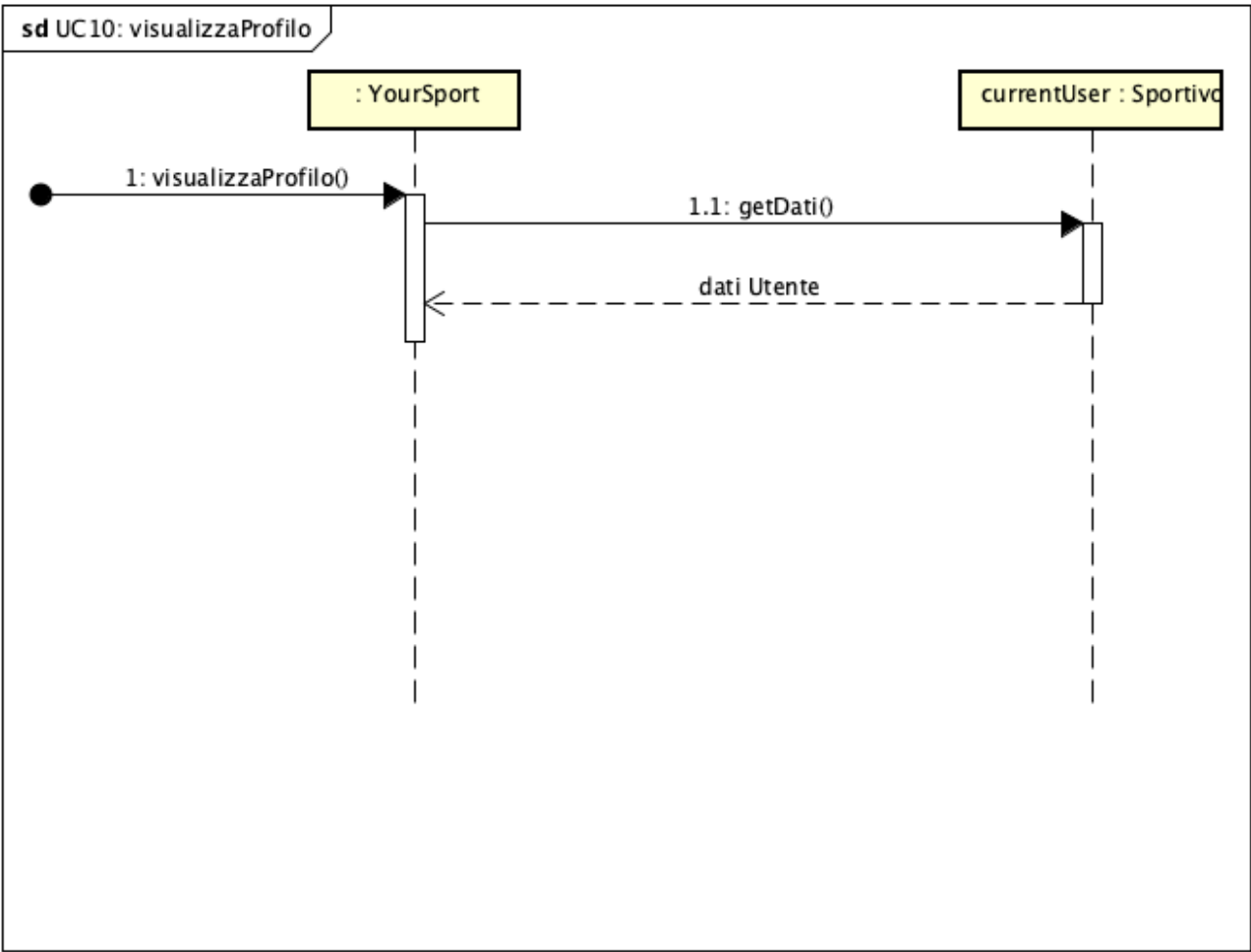
Contratti delle operazioni

-UC10

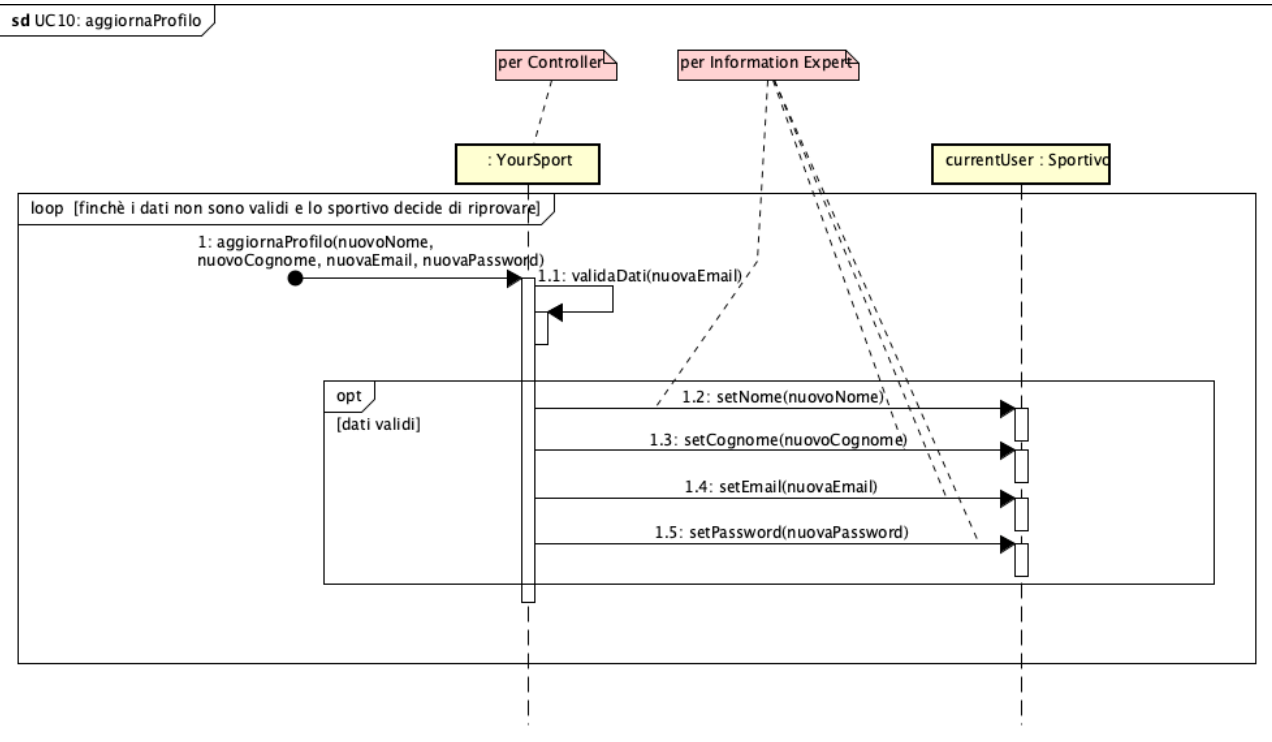
Operazioni	aggiornaProfilo(nuovoNome: String, nuovoCognome: String, nuovaEmail: String, nuovaPassword: String)
Riferimenti	UC10: Gestione profilo personale
Pre-condizioni	Esiste un'istanza di Struttura s associata all'identificatore idStruttura, e il suo attributo isOperativo è attualmente false.
Post-condizioni	-L'attributo nome dell'istanza s è stato aggiornato con il valore nuovoNome. -L'attributo cognome dell'istanza s è stato aggiornato con il valore nuovoCognome. -L'attributo email dell'istanza s è stato aggiornato con il valore nuovaEmail. -L'attributo password dell'istanza s è stato aggiornato con il valore nuovaPassword.

Diagrammi di sequenza

-visualizzaProfilo



-aggiornaProfilo



Testing

Test 1: Visualizzazione Profilo (Utente Loggato)

Descrizione: Simuliamo l'accesso di uno sportivo alla propria area personale per visualizzare i dati anagrafici e le credenziali attuali.

Scopo: Assicurarci che il sistema riconosca correttamente l'utente in sessione e recuperi le informazioni esatte dall'oggetto currentUser, formattandole per la visualizzazione a schermo.

Realizzazione: Il test ripristina lo stato del sistema, effettua il login con un utente fittizio (es. Mario Rossi) e invoca il metodo visualizzaProfilo(). Tramite un'asserzione semplice (assertTrue), verifica che la stringa di testo restituita dal sistema contenga effettivamente il nome dell'utente appena loggato.

Test 2: Aggiornamento Profilo con Successo (Dati Validi)

Descrizione: Simuliamo l'aggiornamento dei dati personali da parte dello sportivo (cambio nome, email, password) inserendo valori validi e non in uso da altri utenti.

Scopo: Verificare che il sistema approvi la modifica e sovrascriva in modo permanente i vecchi attributi all'interno dell'oggetto Sportivo presente in memoria.

Realizzazione: Dopo aver effettuato il login, il test invoca il metodo aggiornaProfilo passando un nuovo set di dati. Il test si conclude con due verifiche dirette (assertTrue): la prima controlla che il metodo restituisca true (esito positivo); la seconda ispeziona direttamente l'oggetto currentUser per confermare che i vecchi dati siano stati fisicamente rimpiazzati da quelli nuovi.

Test 3: Aggiornamento Fallito per Email Duplicata (Estensione 5a)

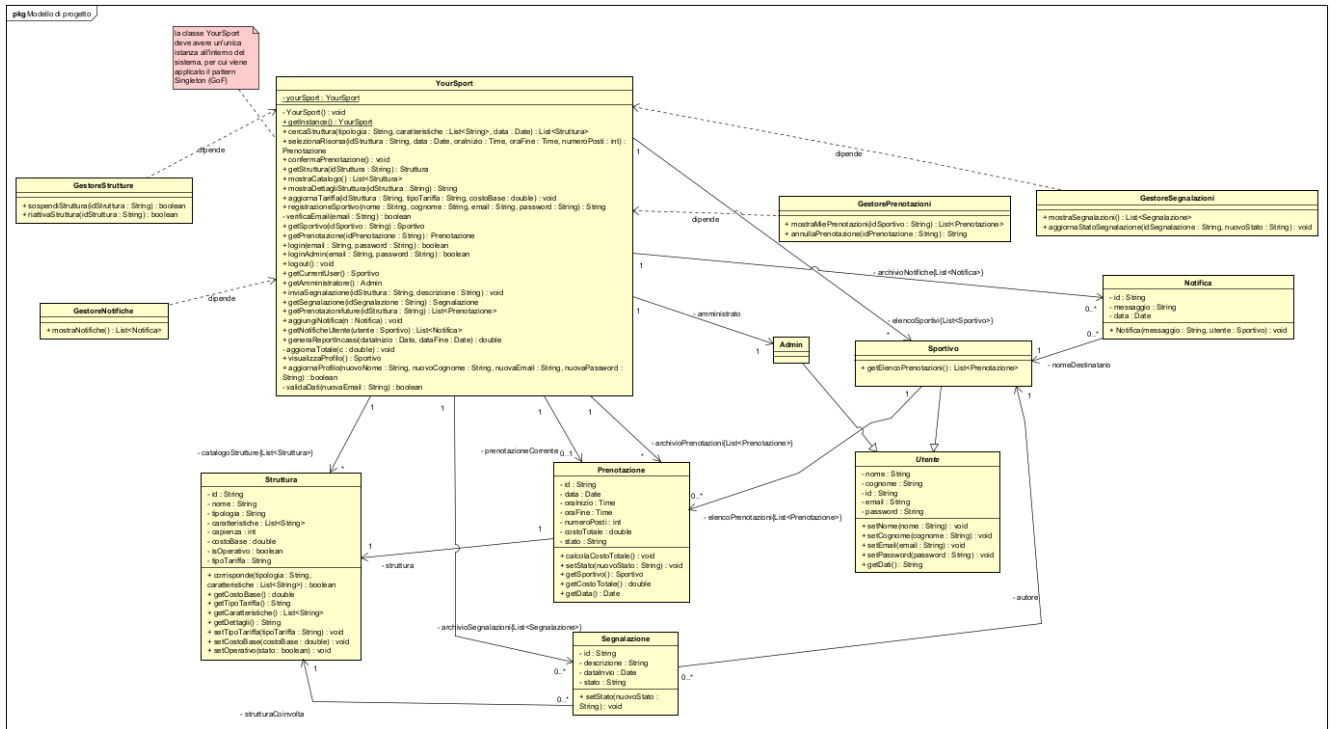
Descrizione: Simuliamo un errore critico di input: l'utente tenta di cambiare la propria email inserendone una già registrata e associata a un altro sportivo presente nel database.

Scopo: Garantire l'integrità dei dati e l'univocità degli account. La Business Logic di validazione deve intercettare il conflitto, bloccare immediatamente la procedura e impedire la corruzione del profilo originale.

Realizzazione: Il test registra un "Utente B" nel sistema per occupare una specifica email. Successivamente, effettua il login con l'"Utente A" e tenta un aggiornamento inserendo l'email dell'Utente B. Il test verifica con successo (assertTrue) che il metodo restituisca false (aggiornamento respinto) e che, ispezionando il profilo dell'Utente A, la sua email originale sia rimasta perfettamente intatta.

3. Diagramma delle Classi

Per quanto riguarda il modello di progetto, i diagrammi di sequenza delle principali operazioni sono stati riportati nella descrizione delle varie iterazioni, a seguire invece, è mostrato il diagramma delle classi che rappresenta la struttura di YourSport.



4. Conclusioni e implementazioni future

4.1 Sintesi del progetto

Lo sviluppo del sistema **YourSport** ha rappresentato un'opportunità concreta per i principi fondamentali dell'Ingegneria del Software. Dalla fase iniziale di ideazione e raccolta dei requisiti, fino all'implementazione dell'ultima iterazione, il progetto è stato guidato dall'obiettivo di creare un'architettura robusta e coesa.

L'utilizzo del linguaggio UML ha permesso di progettare le interazioni in modo chiaro prima ancora di scrivere il codice. L'applicazione rigorosa dei pattern GRASP e dei pattern GoF ha garantito un buon disaccoppiamento tra le responsabilità delle classi. Infine, l'implementazione dei test, supportato dalla stesura di suite JUnit mirate per ogni Caso d'Uso, ha certificato l'affidabilità delle logiche di business più critiche, come la prevenzione dell'overbooking e l'annullamento a cascata durante le emergenze. Il risultato è un motore gestionale solido e pronto per essere scalato.

4.2 Implementazioni future

In un'ottica di rilascio in un ambiente di produzione reale, si possono identificare diverse aree di miglioramento:

- **Evoluzione della Persistenza dei Dati (Database):**

Attualmente, il sistema salva e recupera tutti i dati leggendo e scrivendo su un semplice file locale in formato JSON. Sebbene questa scelta si sia rivelata ottima per la fase prototipale e di testing, uno sviluppo futuro essenziale riguarderà la migrazione verso un vero e proprio DBMS (Database Management System). L'adozione di un database relazionale (es. MySQL), garantirebbe una maggiore scalabilità, permetterebbe di gestire un volume di dati superiore e offrirebbe controlli avanzati sugli accessi concorrenti.

- **Sviluppo di un'App o di un Sito Web:**

Il sistema attuale è completo dal punto di vista logico, ma è sprovvisto di una veste grafica. Un miglioramento futuro consisterà nella creazione di un vero e proprio sito web o di un'applicazione mobile, così che gli utenti possano navigare nel catalogo, prenotare e gestire il proprio profilo comodamente dal telefono o dal computer.